東北大學 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY



Chapter 3 Sequential Logic Design

单选题 1分

下列关于时序逻辑电路与组合逻辑电路的根本区别，描述正确的是?

A 组合逻辑电路必须有时钟信号，而时序逻辑电路可以没有

B 时序逻辑电路的输出仅取决于当前输入，组合逻辑还依赖历史状态

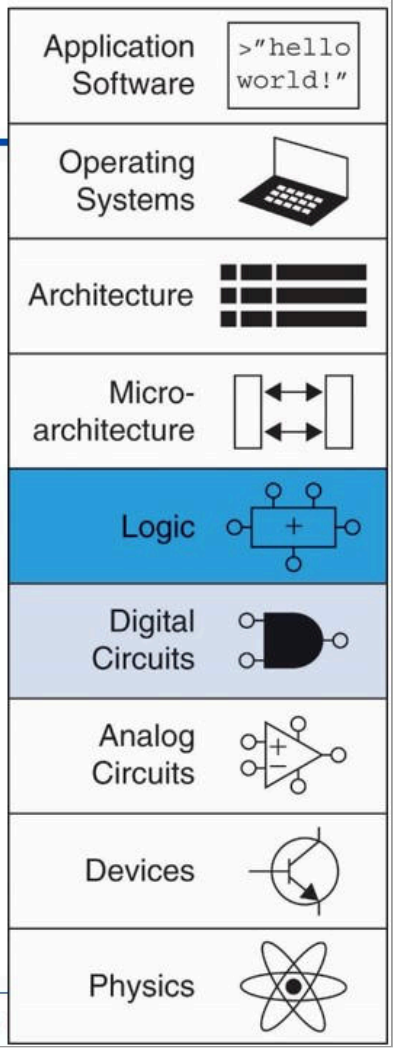
C 时序逻辑电路必须包含存储元件（如触发器），其输出依赖当前输入和电路当前状态

D 两者均不需反馈回路，区别仅在于门电路类型不同

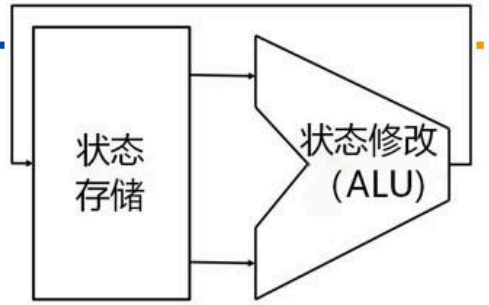
雨课堂 Rain Classroom

Chapter 3 :: Topics

- 引言
- 锁存器 (Latches) 和 触发器 (Flip-Flops)
- 寄存器 (Register) 和 寄存器文件 (Register File)
- 同步逻辑设计 (Synchronous Logic Design)
- 有限状态机 (Finite State Machines)
- 时序逻辑的时序 (Timing of Sequential Logic)
- 并行 (Parallelism)



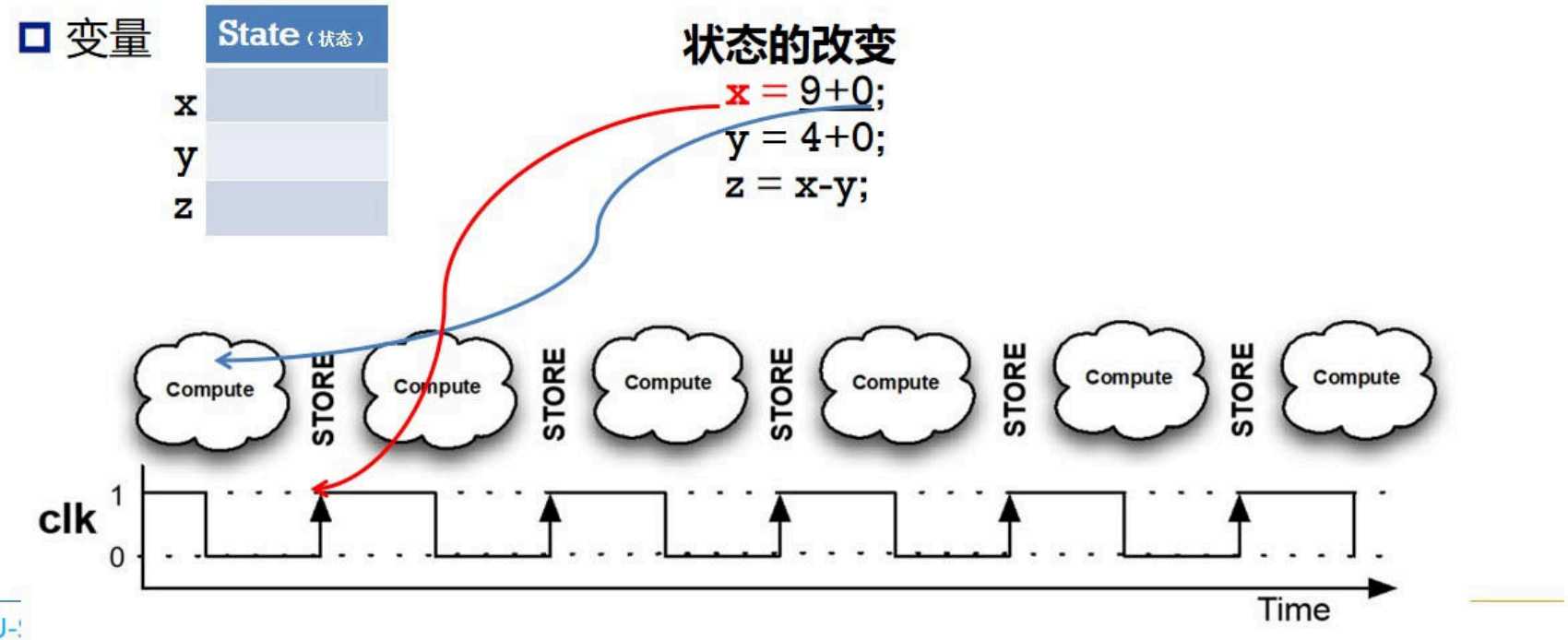
状态 – 计算机的核心概念



- 抽象来说，计算机就**反复在做 2 件事儿**：
  (1) 状态的存储（**要理解怎么存储这些“bit”位**）
  (2) 状态的改变（**用布尔代数来改变“状态位”**）
- **状态** = 系统在某一瞬间先前的输入信息，或者先前输入的一部分信息。
- 硬件层面
  - 寄存器状态：寄存器中每一位 0/1，都是运算的临时状态。
  - 内存（RAM）状态：程序运行时变量、数组、栈、堆，全部是内存状态。
    - 举例子：int a = 10，内存某地址存数字 10，这就是变量 a 的状态；a 变成 20，就是状态修改。

抽象的说，计算机就做 2 件事儿

- 每个时钟周期，计算系统都会存储更新后的状态信息
- 系统时钟决定每一行代码的执行时序



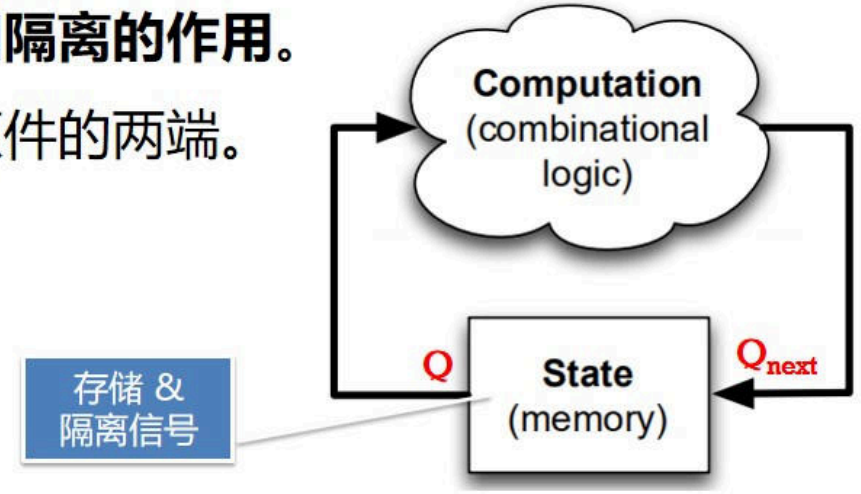
引言

- 时序逻辑的输出不仅仅依赖当前的输入，还依赖之前的输入值
  - 所以**必须有存储部件**，能存储之前的状态

- 有必要明确几个概念：
  - **状态变量**：能够解释电路未来行为的所需要的有关过去的所有信息。
  - **锁存器**和**触发器**：存储一位状态变量的存储元，也是最简单的时序逻辑电路。
  - **同步时序电路**：组合逻辑+触发器

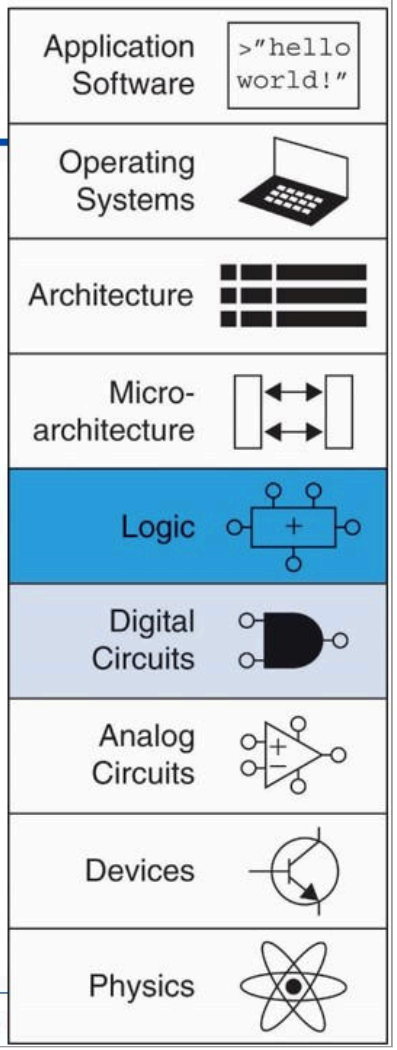
时序逻辑电路 Sequential Circuits

- 时序逻辑电路，指定了事件的顺序（通过状态的变迁来体现）。
- 正因为有顺序，所以需要存储状态，且通过输出到输入的反馈来存储状态信息以及状态的变迁。
- 这里的**记忆原件起到了状态存储和隔离的作用。**
- 当前状态和下一状态分处在记忆原件的两端。



Chapter 3 :: Topics

- 引言
- **锁存器 (Latches) 和 触发器 (Flip-Flops)**
- 寄存器 (Register) 和 寄存器文件 (Register File)
- 同步逻辑设计 (Synchronous Logic Design)
- 有限状态机 (Finite State Machines)
- 时序逻辑的时序 (Timing of Sequential Logic)
- 并行 (Parallelism)



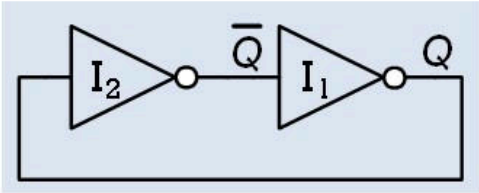
状态元件 State Elements

- 电路的当前状态会影响它新的状态
- 通过**状态元件**存储状态，状态元件有：
  - 双稳态电路 (Bistable circuit)
  - SR锁存器 (SR Latch)
  - D锁存器 (D Latch)
  - D触发器 (D Flip-flop)

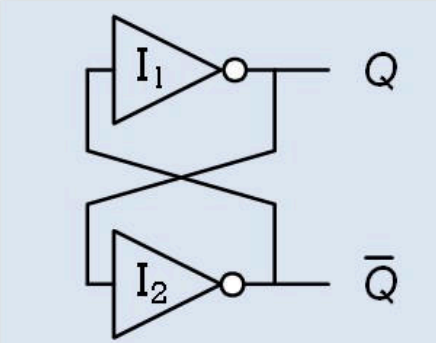
雨课堂 Rain Classroom

双稳态电路 Bistable Circuit

- 双稳态元件是存储器件的基本模块，这个元件有**两种稳定的状态**：Q, Q'
- 一对反相器就可以组成一个简单的双稳态元件。
  - 这两个反相器交叉耦合，即I_1的输入是I_2的输出取反
  - 这个电路没有输入，但可以有两个输出

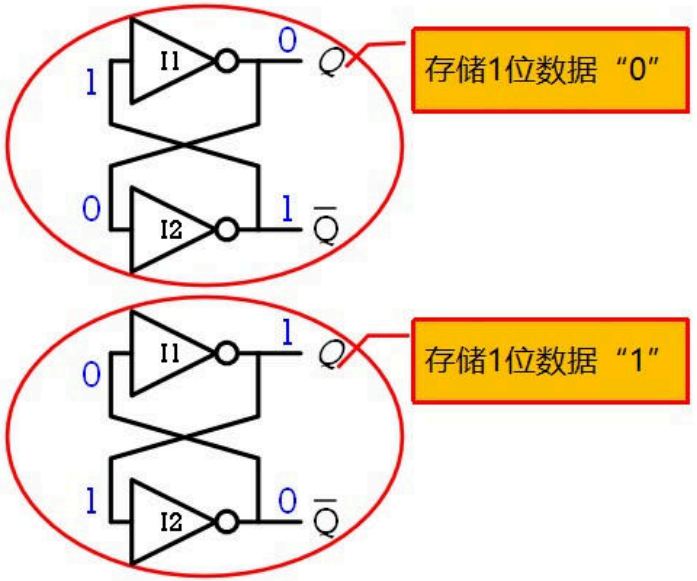


- 将图重新画了一下，看起来具有对称性。



双稳态电路分析

- 考虑两种可能得情况:
  - $Q = 0$: then $Q' = 1$, $Q = 0$ (状态稳定)
  - $Q = 1$: then $Q' = 0$, $Q = 1$ (状态稳定)

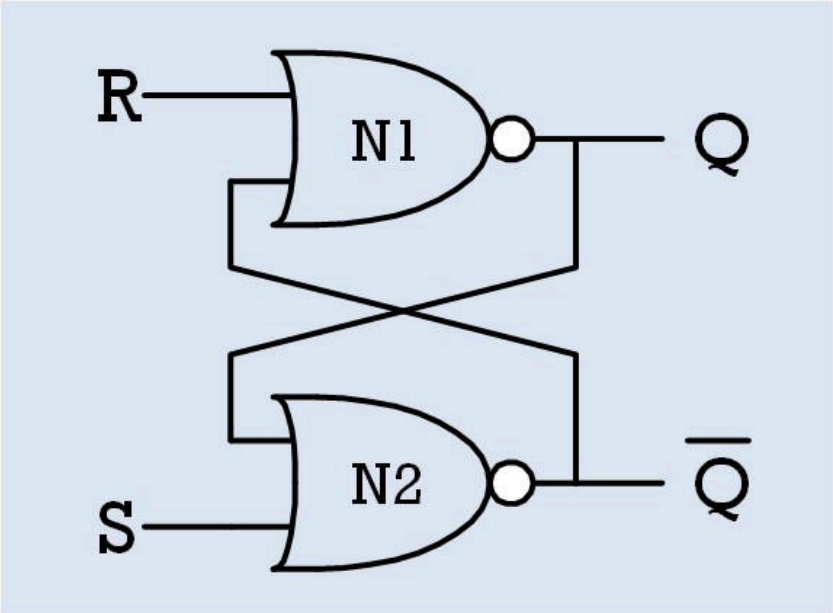


- 存储了1 bit 状态在 Q 这个状态变量中
- 但是这个逻辑结构中**没有输入来控制初始状态或状态的翻转**

SR锁存器 Latch

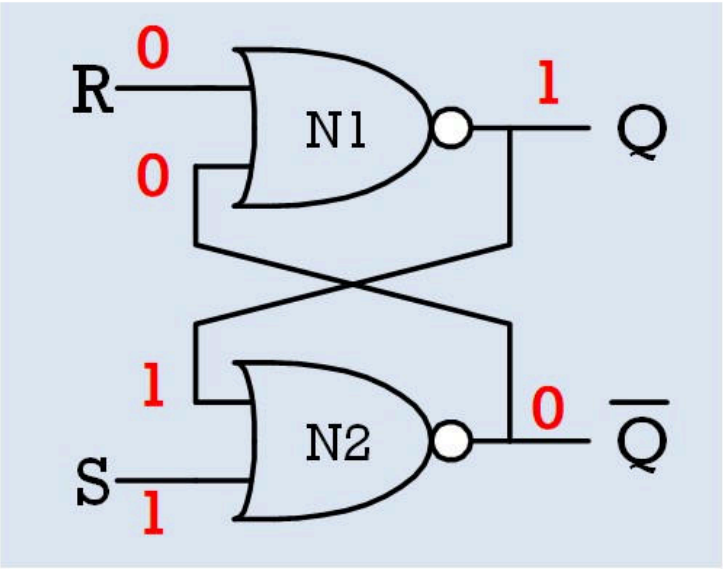
- SR (Set/Reset)

- 考虑四种可能的情况:
  - S = 1, R = 0
  - S = 0, R = 1
  - S = 0, R = 0
  - S = 1, R = 1



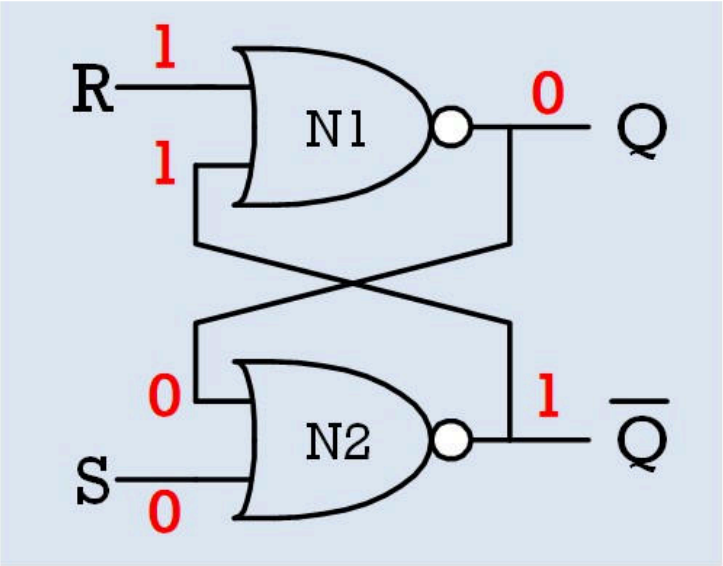
SR锁存器分析

- S = 1, R = 0: then Q' = 0 and Q = 1
  - 置位状态（Set）
  - 当Set=1，R=0；N1和N2两个或非门中N2确定会翻转到0，N1还得等另外一个引脚的状态。



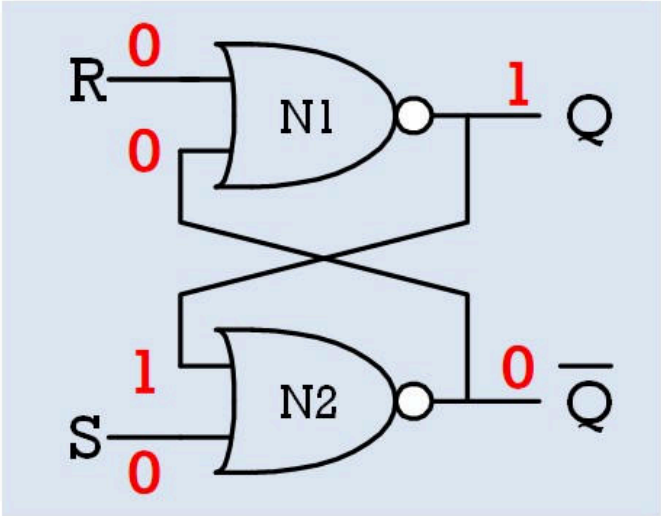
SR锁存器分析

- S = 0, R = 1: then Q = 0 and Q' = 1
  - 复位状态（Reset）
  - 当Reset=1，S=0；N1和N2两个或非门中，N1确定会翻转到0，N2还得等另外一个引脚的状态。

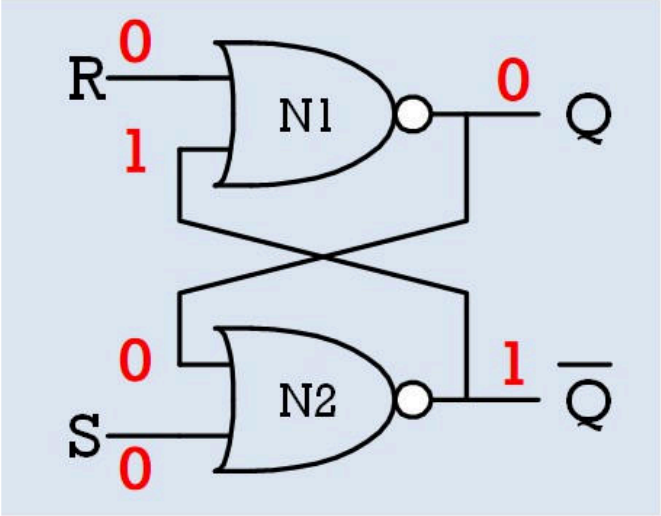


SR锁存器分析

- S = 0, R = 0: then $Q = Q_{prev}$
  - 保持状态（Consistent）



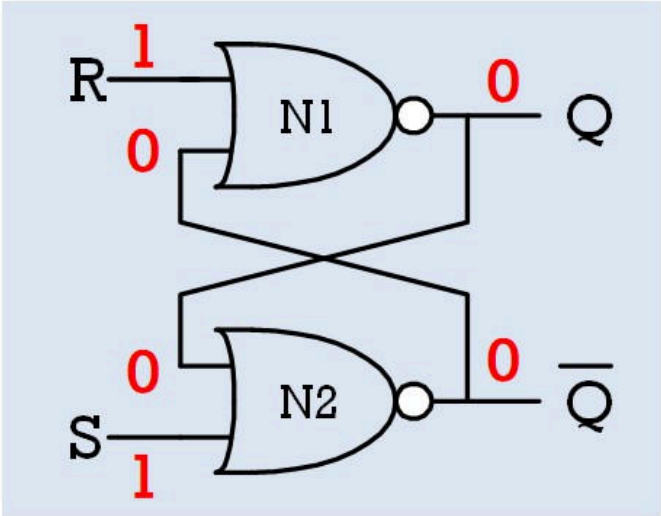
$Q_{prev} = 1$

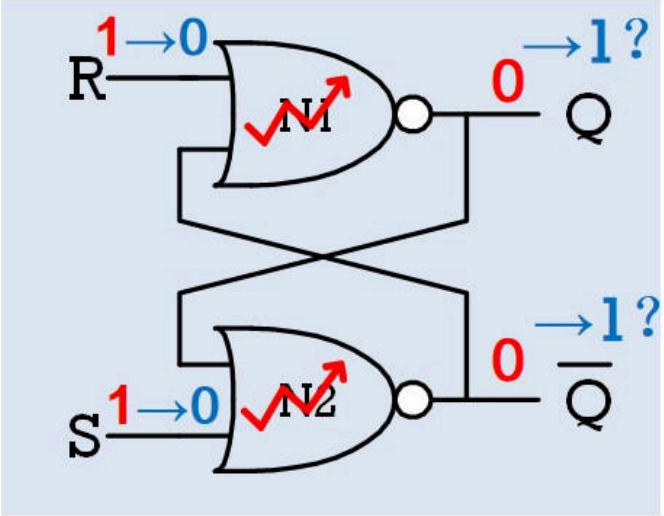


$Q_{prev} = 0$

SR锁存器分析

- S = 1, R = 1: then Q = 0 and Q' = 0
  - 无效/约束状态（Invalid State）

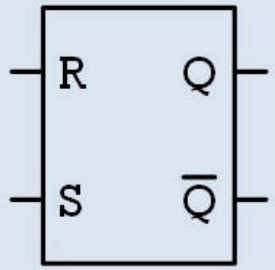




SR锁存器小结

- 一个SR锁存器存储 1 bit 的状态 (Q)

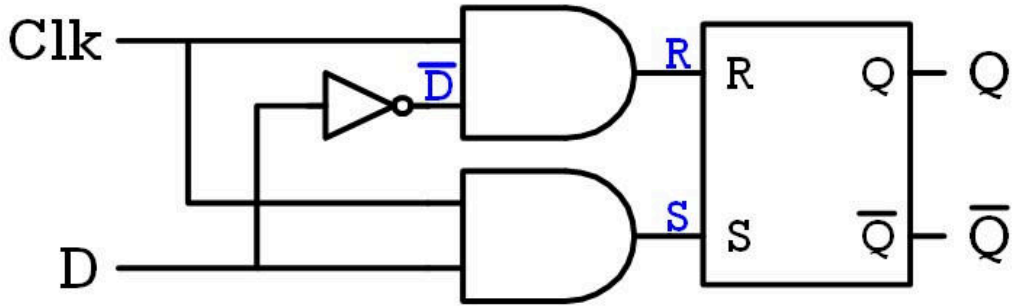
SR Latch Symbol



- S = 1, R = 0: then Q = 1, 置位 (Set)
- S = 0, R = 1: then Q = 0, 复位 (Reset)
- S = 0, R = 0: then $Q = Q_{prev}$, 存储/保持 (Memory)
- S = 1, R = 1: 无效状态 (Invalid State)

D锁存器 Latch

- 其内部电路：



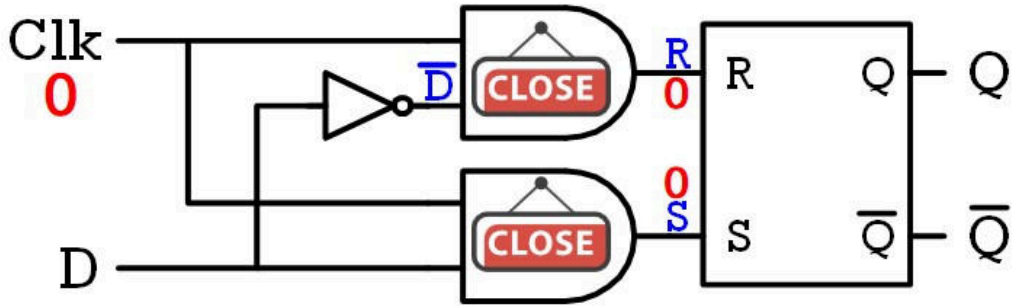
- 功能表

Clk	*D*	$\overline{D}$	*S*	*R*	*Q*	$\overline{Q}$
0	×					
1	0					
1	1					

雨课堂 Rain Classroom

D锁存器

- 其内部电路：

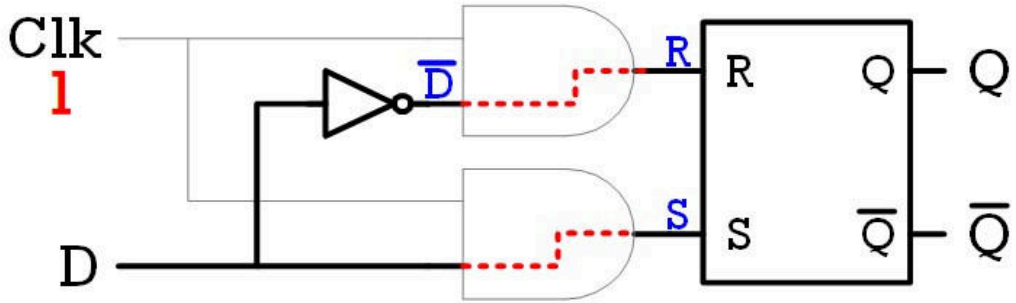


- 功能表

Clk	*D*	$\overline{D}$	*S*	*R*	*Q*	$\overline{Q}$
0	×	×	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0					
1	1					

D锁存器

- 其内部电路：



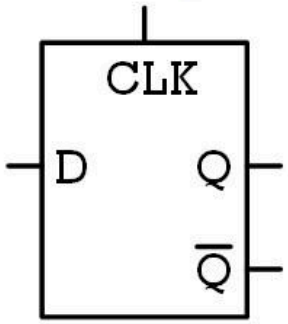
- 功能表

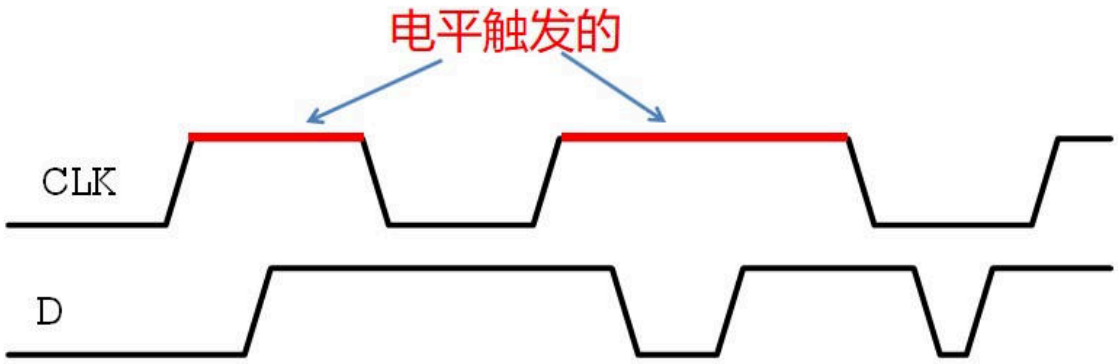
Clk	*D*	$\overline{D}$	*S*	*R*	*Q*	$\overline{Q}$
0	×	×	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

D锁存器小结

- 避免SR锁存器的那种无效输入
- 当 CLK = 1， D直通向Q (D锁存器看起来是透明transparent的)
- 当 CLK = 0， Q保持前一状态 (D锁存器有叫阻塞opaque的)

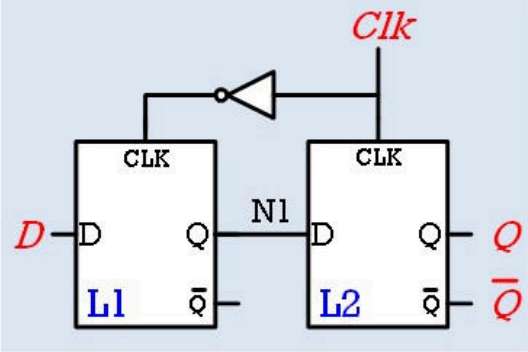
D Latch Symbol



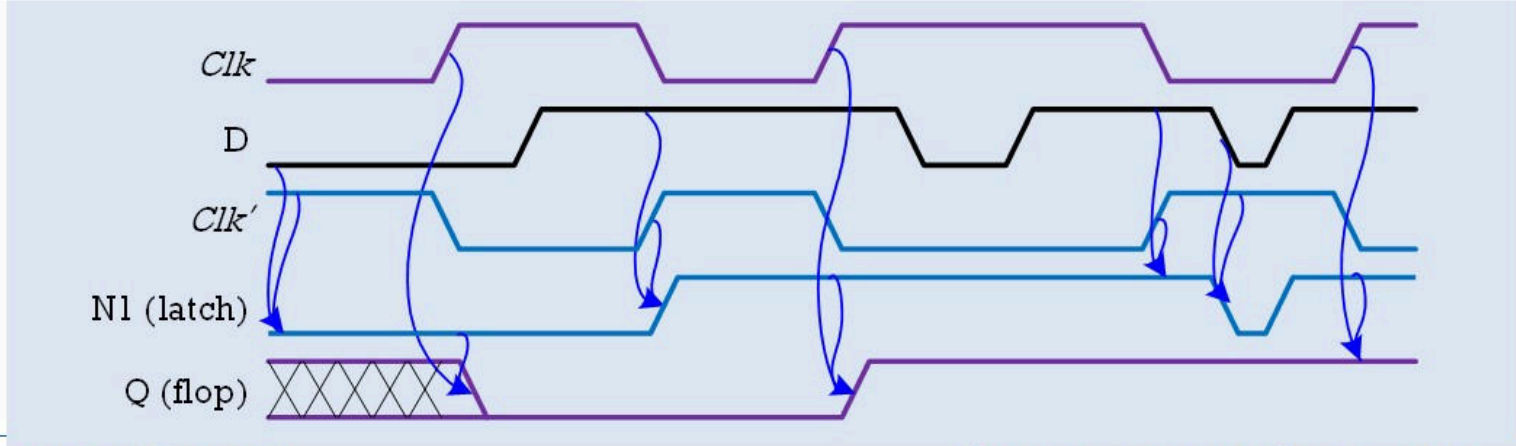


D触发器 Flip-Flop

- 其内部电路：

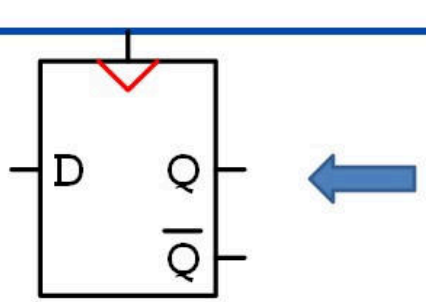


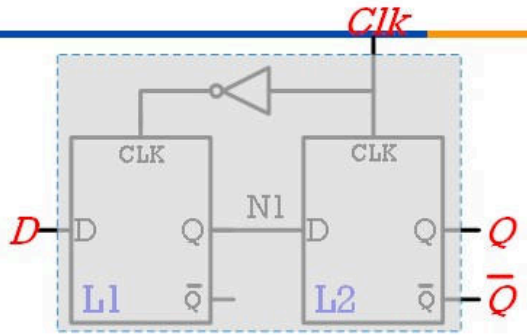
- 时序图：



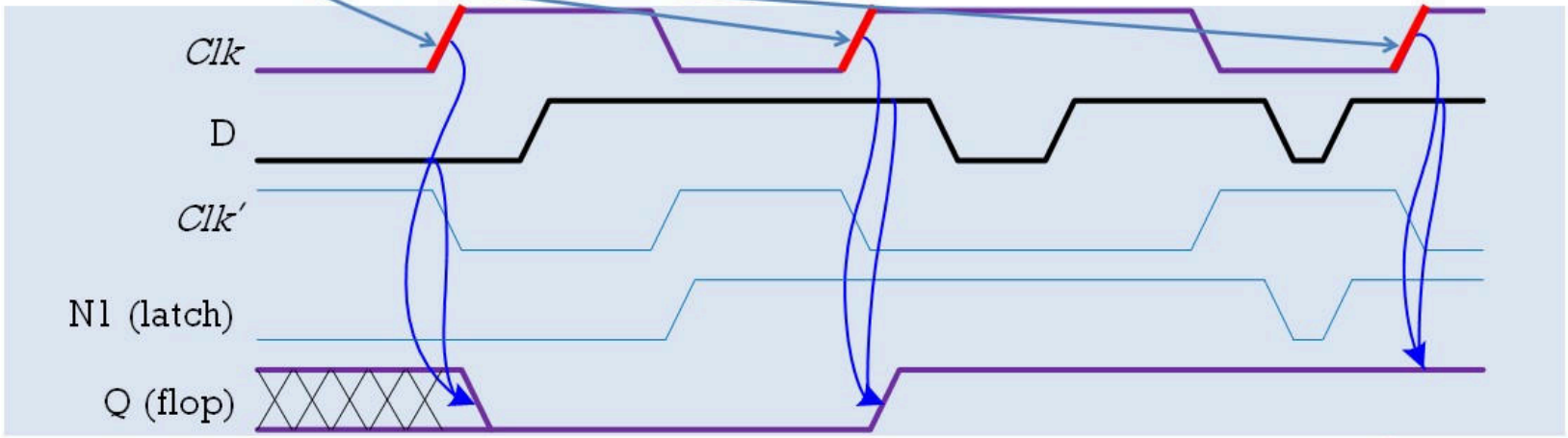
D触发器

- D触发器的符号图

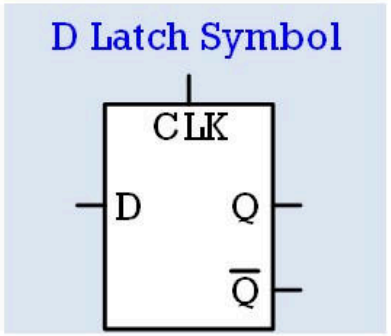


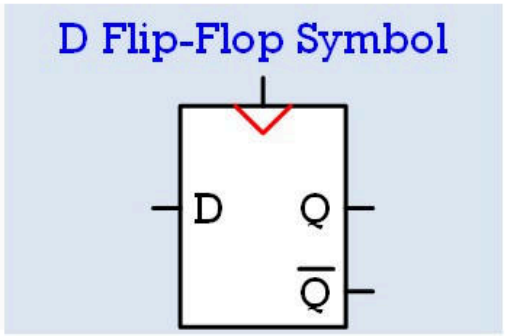


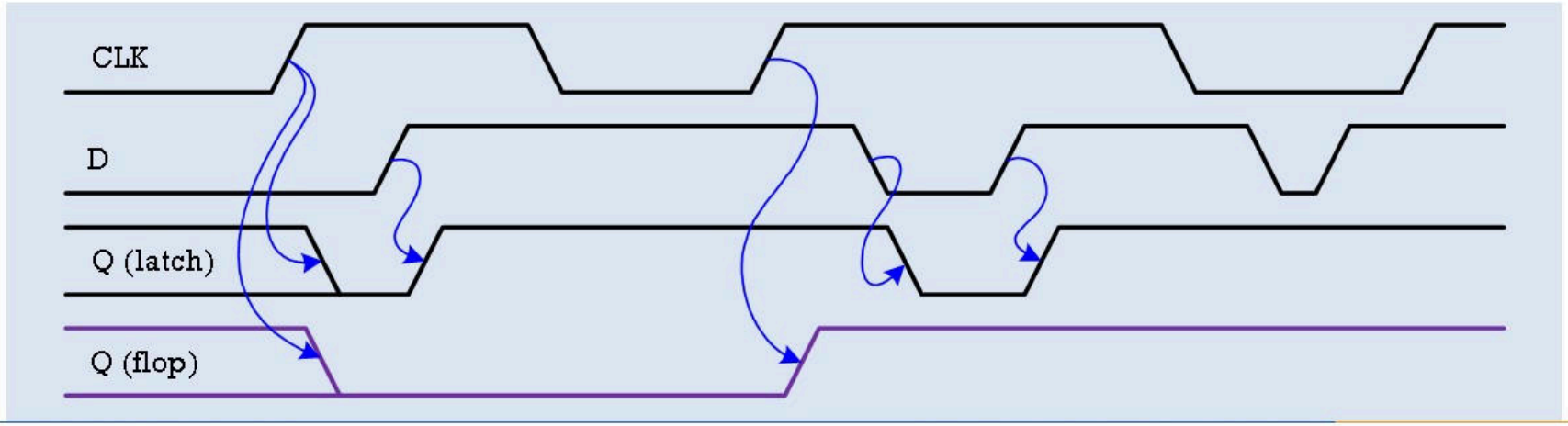
沿触发的edge-triggered



D锁存器 vs. D触发器 对比

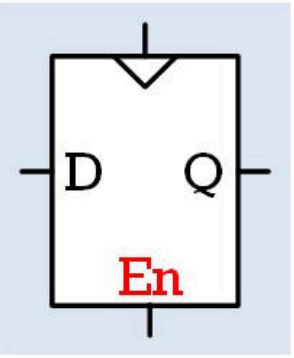






带使能端的D触发器

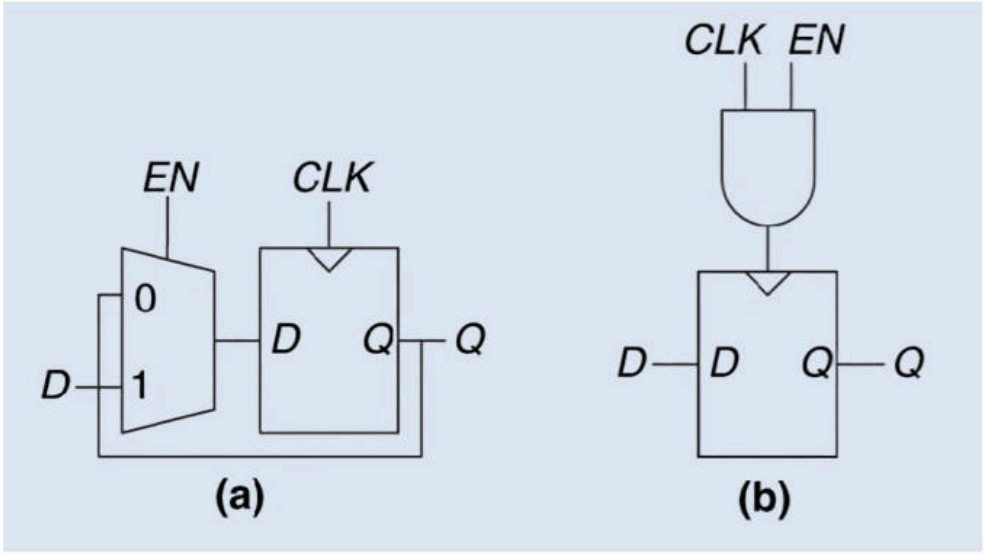
- 增加了一个使能端 (EN)，用于确定在时钟沿是否载入新数据 (D)
  - 当 EN = 1: 带使能端的触发器和普通D触发器一样
    - 在时钟上升沿采集输入D的状态，存储并在Q端体现
  - 当 EN = 0: 带使能端的触发器忽略时钟信号
    - 触发器保持原来的状态不变



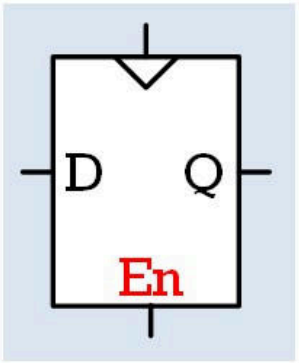
- **同步时序逻辑**中一个CLK信号会控制很多寄存器，控制它们被写入的时刻；
- CLK频率往往较高，实际工作中，我们不会高频的同时写入多个寄存器，当我们希望在特定的时间，而不是每个时钟上升沿都对所有寄存器载入新数据时，使能端EN 就非常有用了。
- 例如，上课铃声(同比CLK)是固定的，但是不是真的一打铃全体学生都上课，还要看课表(同比EN)

带使能端的D触发器的实现

- 下面给出用一个D触发器增加额外的门，组成一个带使能端的触发器的两种方法。



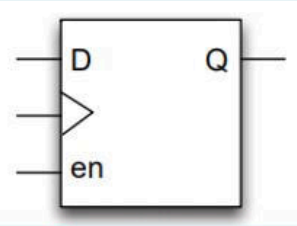




单选题 1分

带使能端的D触发器

- 依据如下时序图判断 x、y 时刻的D触发器状态

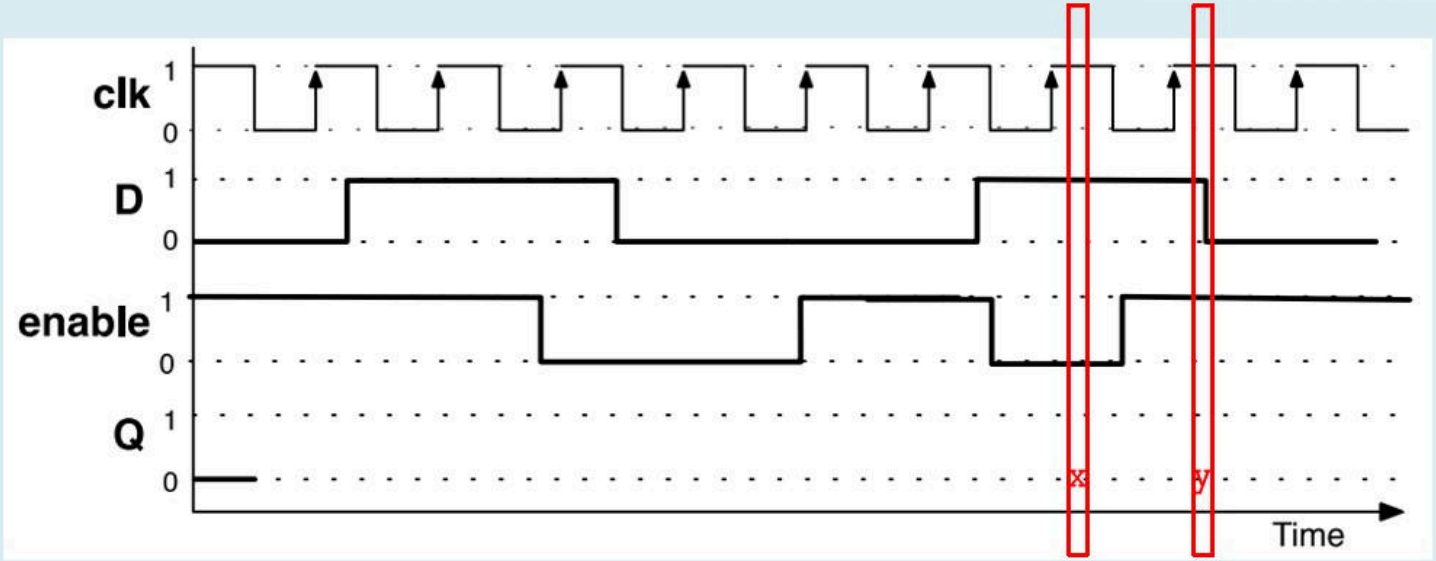


A 00

B 01

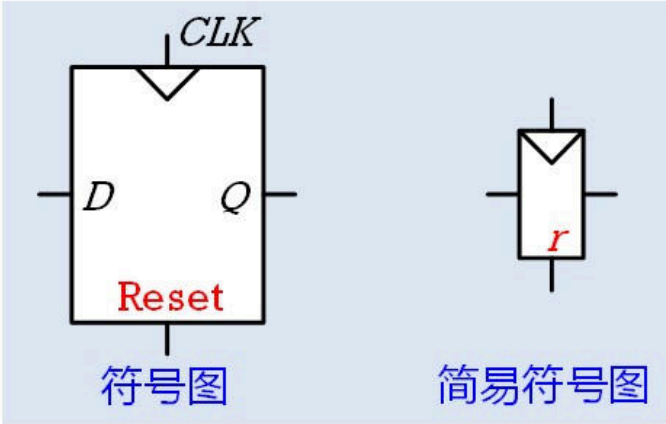
C 10

D 11



带有复位端的D触发器

- 输入端口: CLK, D, Reset

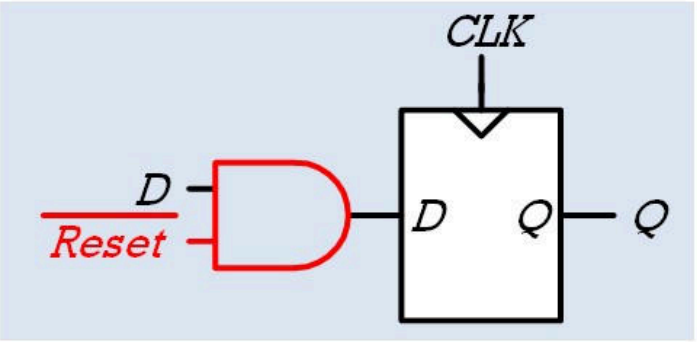


- 功能:
  - 当 Reset = 1: 无论D值为何，Q 强制复位为 0
  - 当 Reset = 0: 触发器和普通D触发器一样

雨课堂 Rain Classroom

带有复位端的D触发器的实现

- 两种类型:
  - 同步复位(Synchronous)：只在时钟上升沿复位
    - 其实现的内部电路:

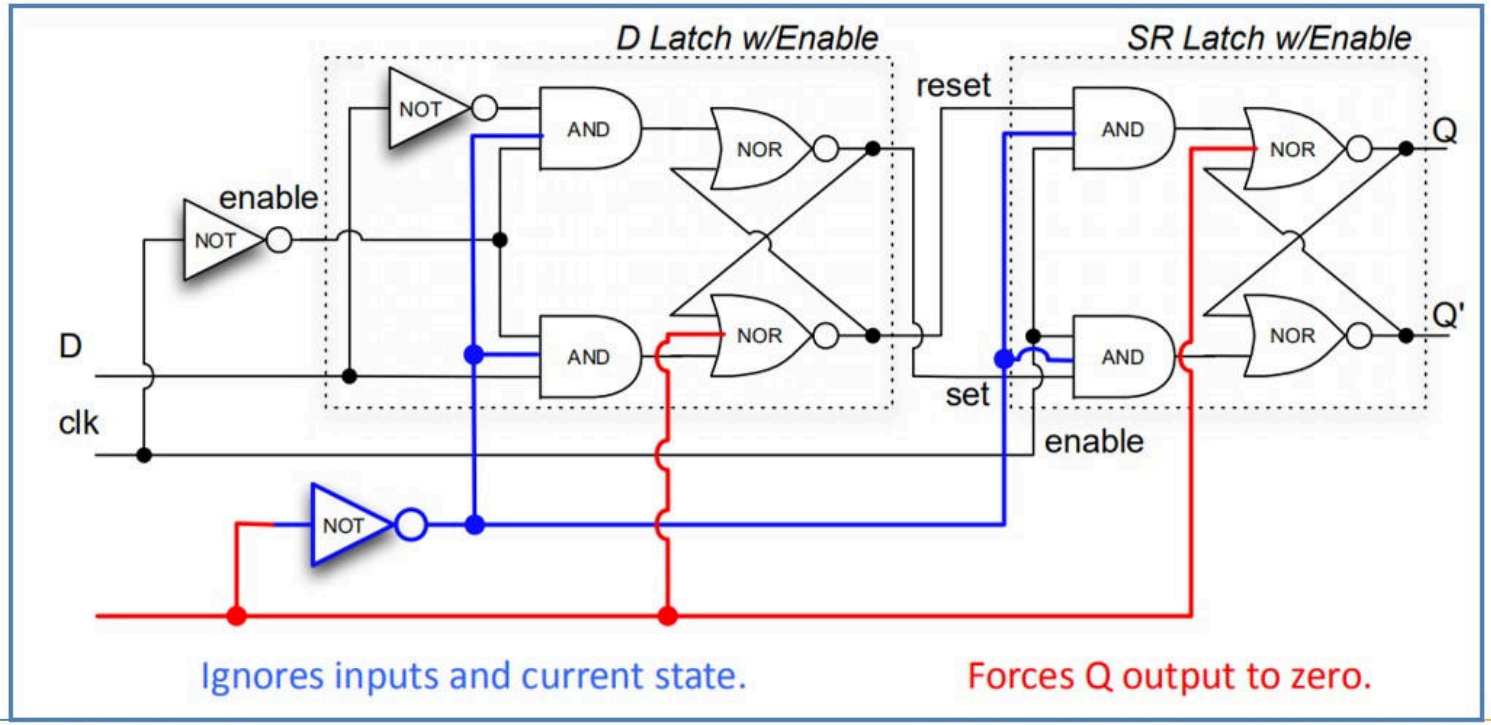


  - 异步复位(Asynchronous)：当 Reset = 1时，立即复位。
    - 异步复位的触发器需要修改D触发器的内部逻辑电路，实现逻辑比较复杂

带有复位端的D触发器的实现

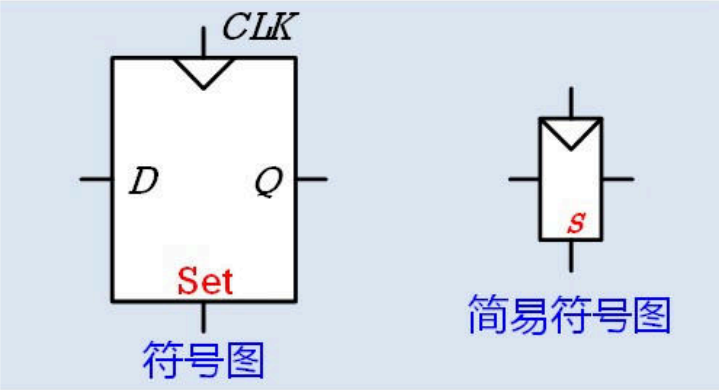
不要求掌握

- 异步复位的D触发器的实现



带有置位端的D触发器

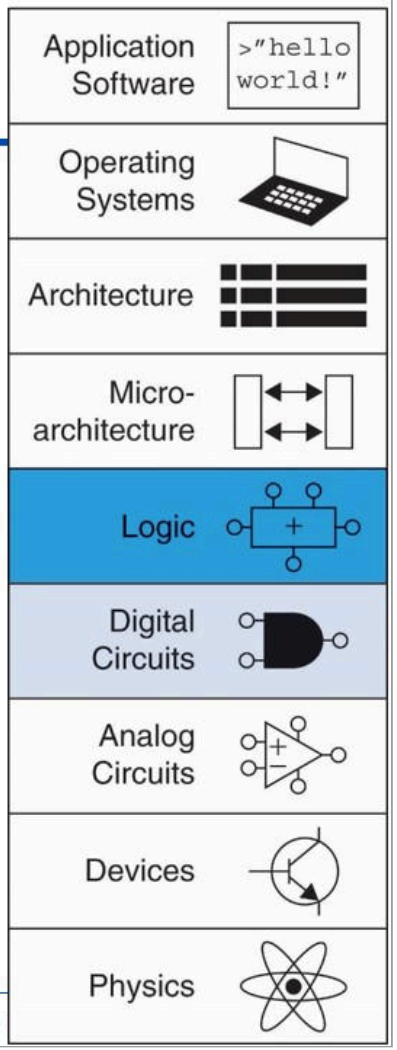
- 输入端口: CLK, D, Set



- 功能:
  - 当 Set = 1: 无论D值为何，Q 端强制置位成 1
  - 当 Set = 0: 触发器和普通D触发器一样

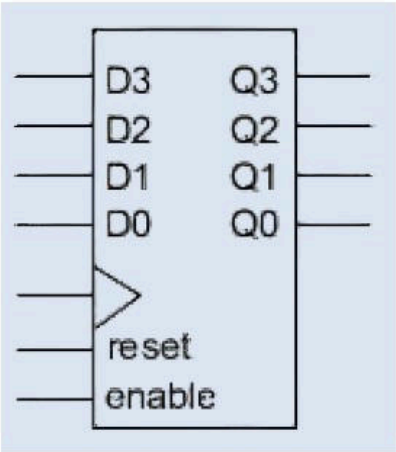
Chapter 3 :: Topics

- 引言
- 锁存器 (Latches) 和 触发器 (Flip-Flops)
- **寄存器 (Register) 和 寄存器文件 (Register File)**
- 同步逻辑设计 (Synchronous Logic Design)
- 有限状态机 (Finite State Machines)
- 时序逻辑的时序 (Timing of Sequential Logic)
- 并行 (Parallelism)

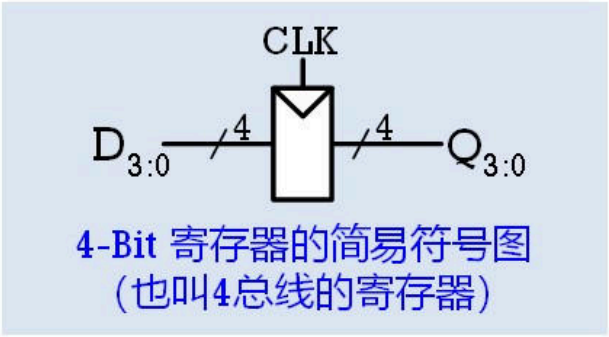


使用多个触发器构建寄存器

- 寄存器是组成时序电路的关键结构。
- 例如，4-bit **寄存器**register 由4个 D 触发器实现



4-Bit 寄存器功能图

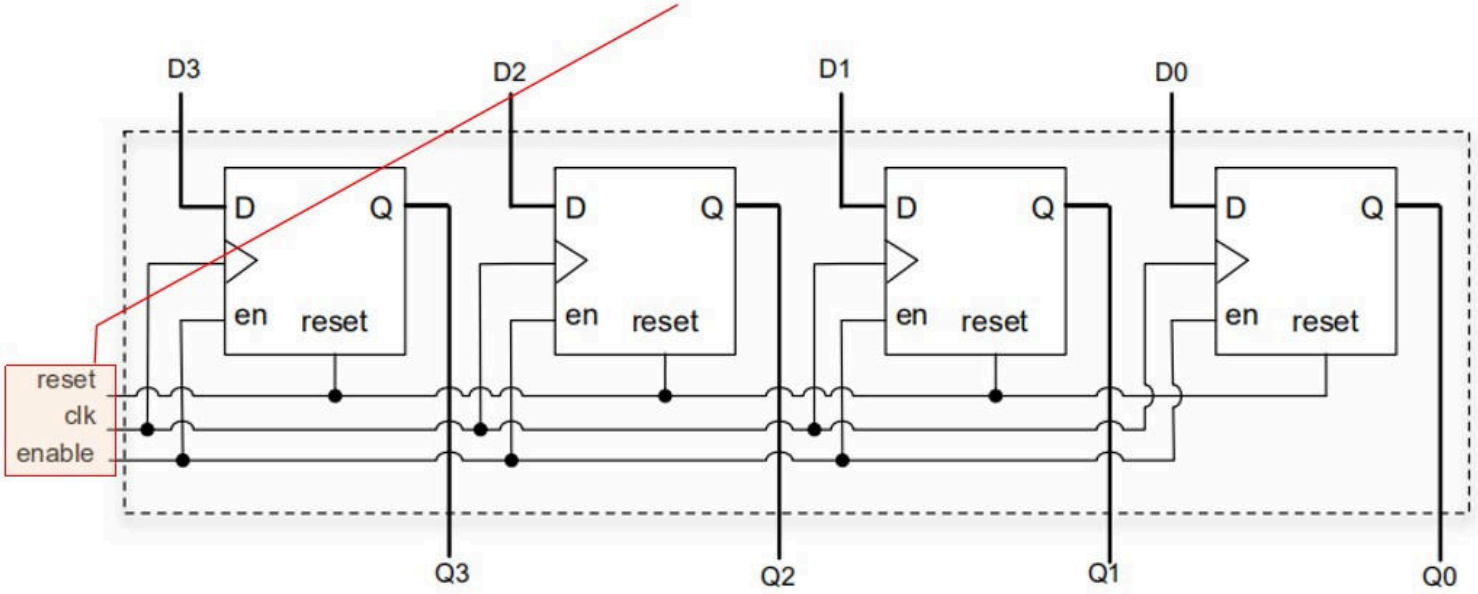


4-Bit 寄存器的简易符号图
（也叫4总线的寄存器）

雨课堂 Rain Classroom

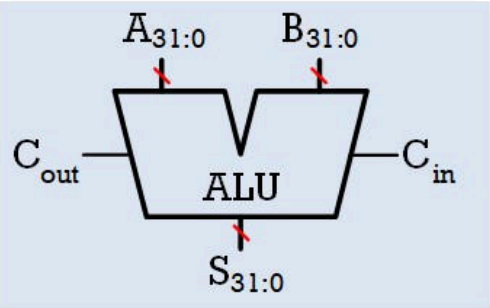
使用多个触发器构建寄存器

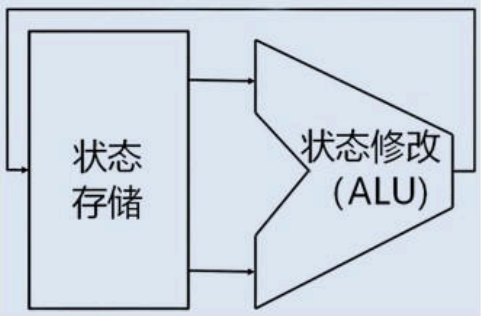
- 4-bit 寄存器实现原理
  - D触发器的所有控制信号都**使用相同的输入**



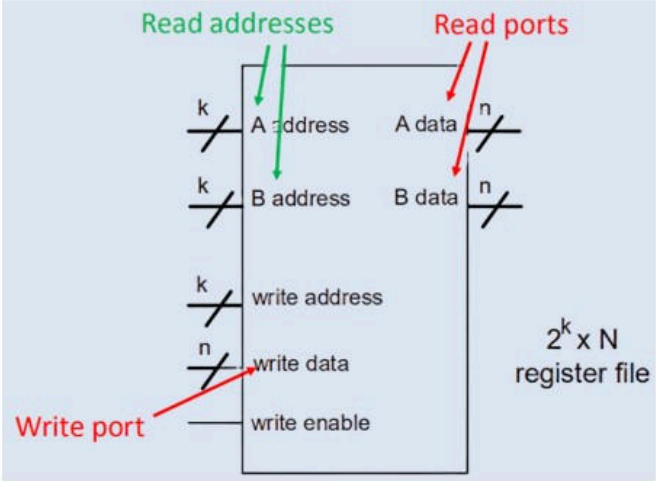
寄存器文件/寄存器堆 Register File

- MIPS架构的寄存器文件支持单周期内的多操作数操作
  - 具有 2 个读端口, 所以可以同时读取两个操作数的数值
  - 1个写入端口



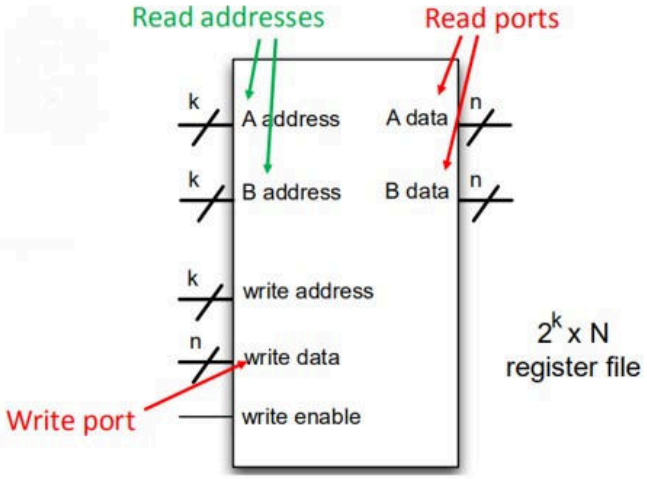


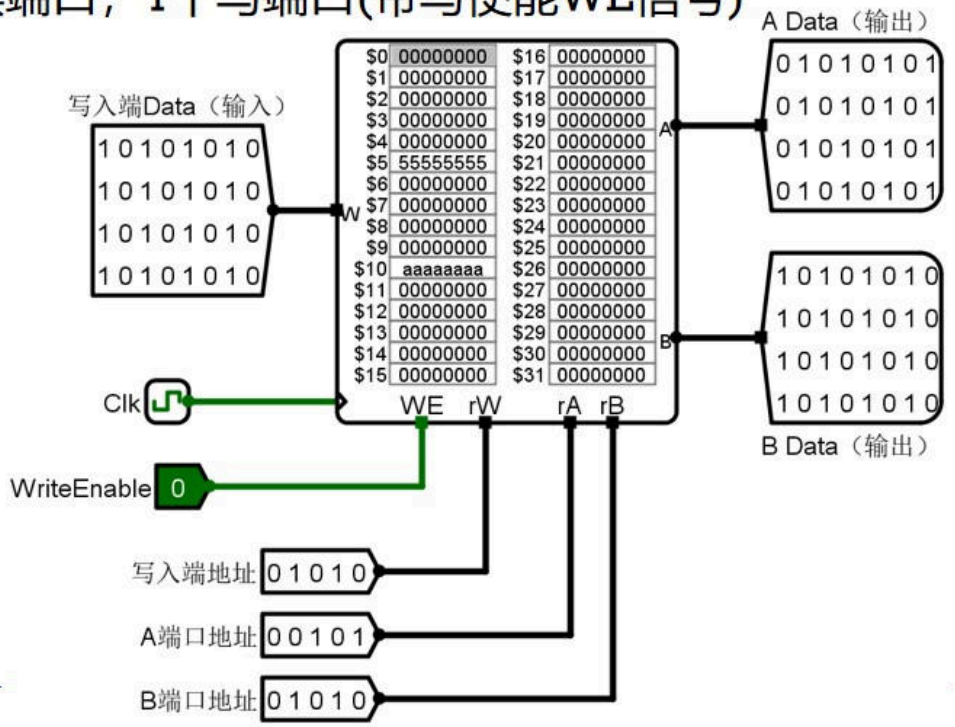
ADD $3, $1,$2



寄存器文件/寄存器堆 Register File

- Logisim中有仿真的MIPS寄存器文件
  - 32个32-bits的寄存器组，2个读端口，1个写端口(带写使能WE信号)

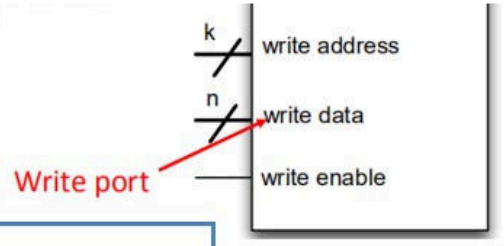


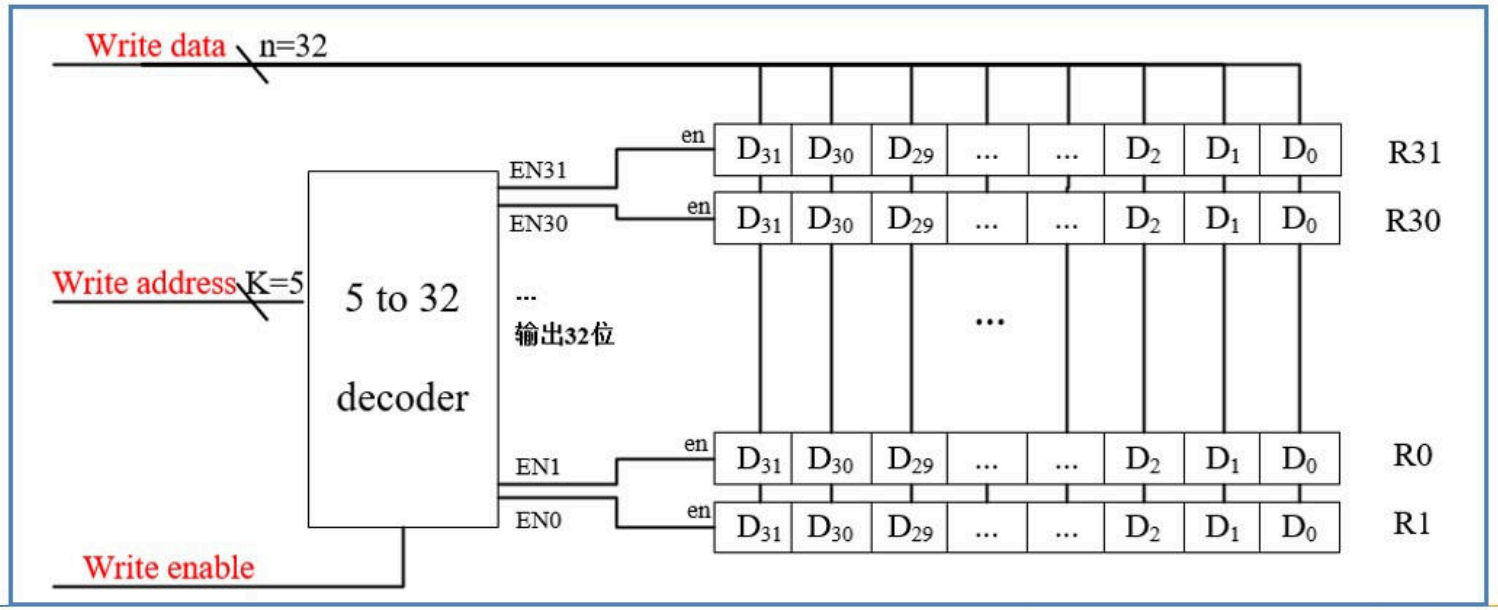


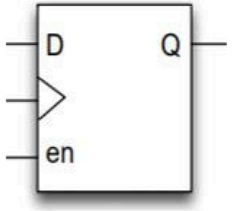
NEU-SWC-A0801040040

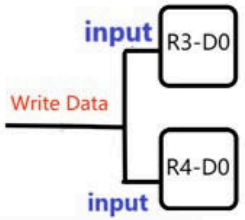
寄存器文件原理

- 写入端口的内部电路
  - 包括5-bit写入地址总线、32-bit写入数据、1-bit写使能信号



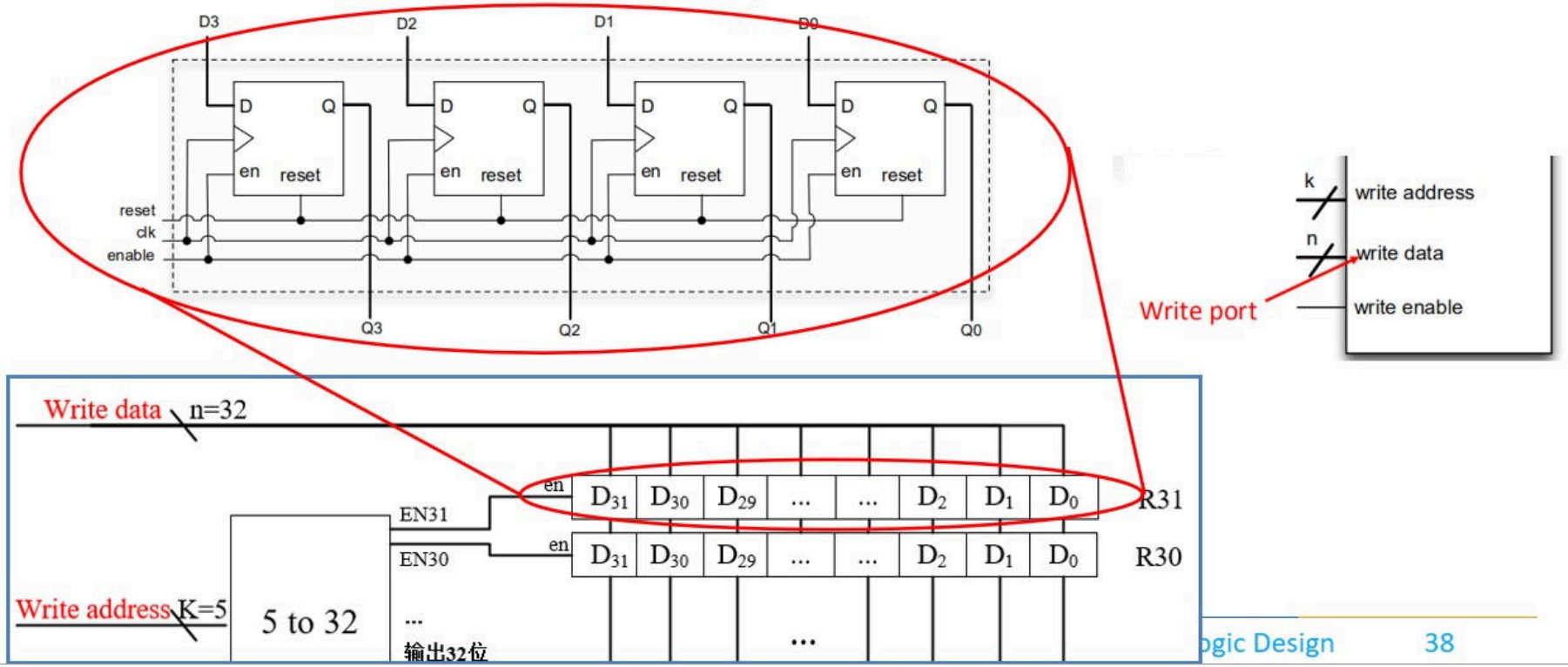






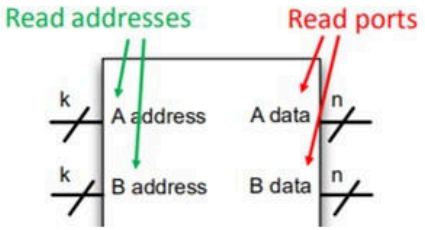
寄存器文件原理

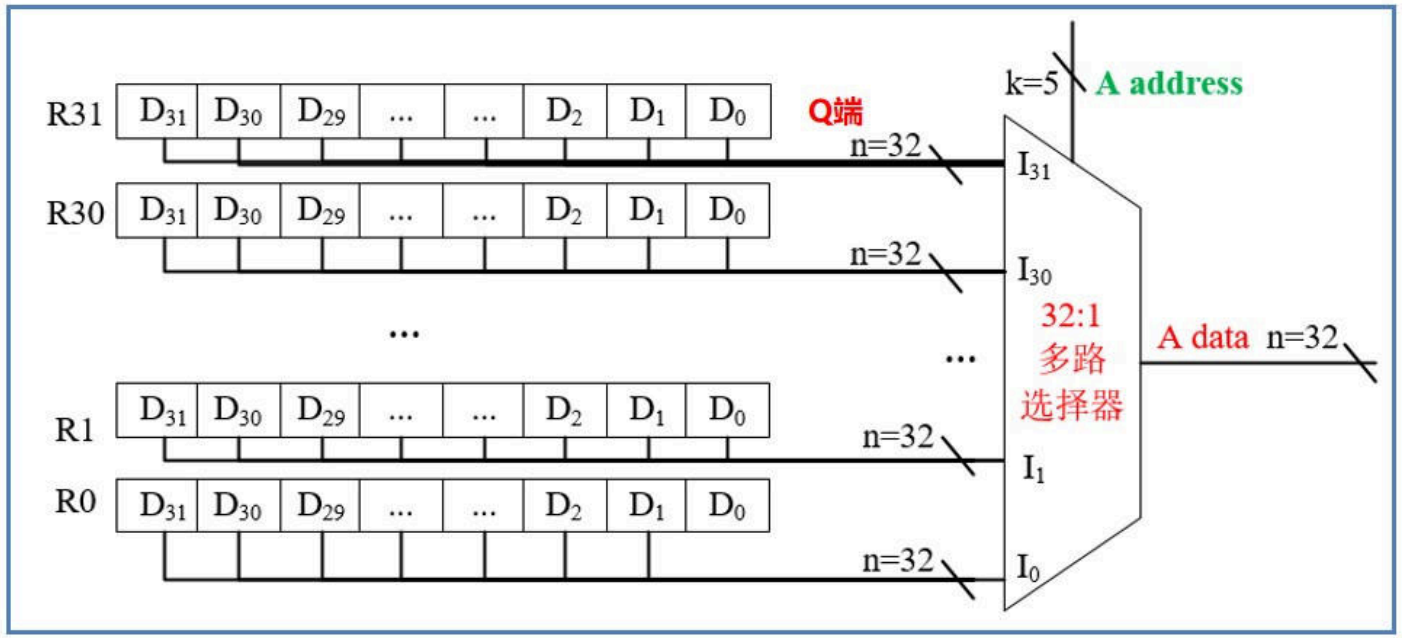
- 写端口对**写地址(Write address)**进行**译码(decode)**，确保**仅对唯一的**一个寄存器执行写入操作（每个时钟周期执行一次）。

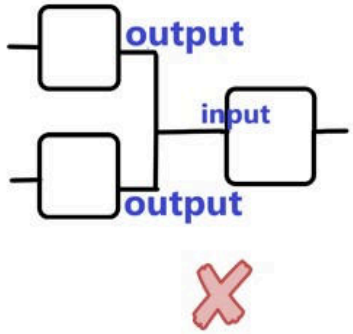


寄存器文件原理

- 读端口利用各自的**读地址(Read address)**选出**一个寄存器**的数据向外输出，该过程持续有效（即随时可读）。







单选题 1分

在寄存器文件的逻辑结构中，二进制**译码器**控制哪些操作?

A 选择从哪个寄存器读取

B 选择向哪一个寄存器写入

C 以上两者

D 两者都不是

雨课堂 Rain Classroom

单选题 1分

- 我们需要设计一块 RAM，能够存储 128 个 64-bit 字，包含 1 个写端口和 3 个读端口。
  - 该 RAM 的写端口需要多少根地址线?

A 1

B 6

C 7

D 64

E 128

雨课堂 Rain Classroom

单选题 1分

- 我们需要设计一块 RAM，能够存储 128 个 64-bit 字，包含 1 个写端口和 3 个读端口。
  - 每个读端口需要多少位数据线?

A 1

B 6

C 7

D 64

E 128

雨课堂 Rain Classroom

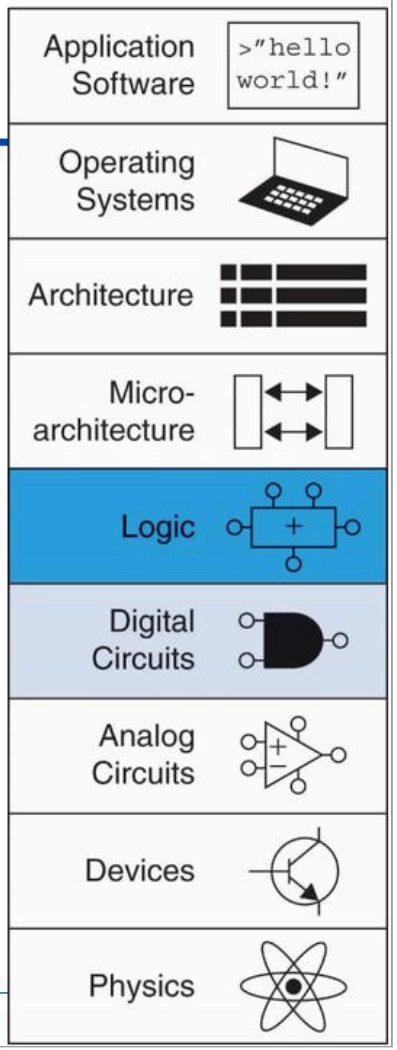
第八堂课结束了

有问题同学们可以提问讨论

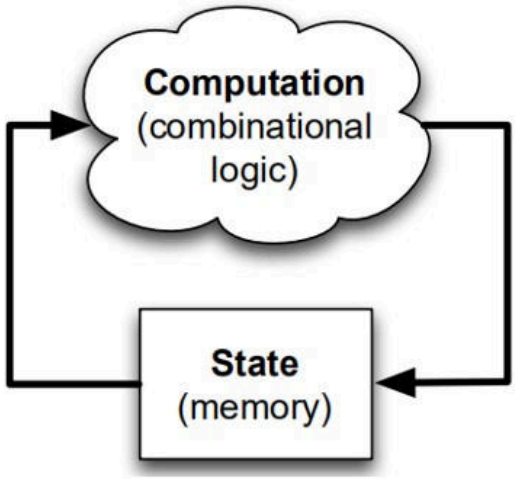
雨课堂 Rain Classroom

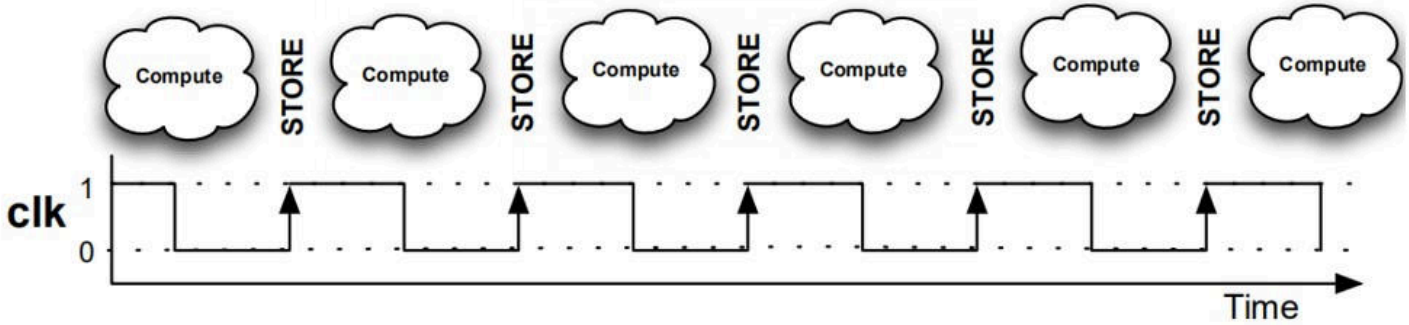
Chapter 3 :: Topics

- 引言
- 锁存器 (Latches) 和 触发器 (Flip-Flops)
- 寄存器 (Register) 和 寄存器文件 (Register File)
- **同步逻辑设计 (Synchronous Logic Design)**
- 有限状态机 (Finite State Machines)
- 时序逻辑的时序 (Timing of Sequential Logic)
- 并行 (Parallelism)



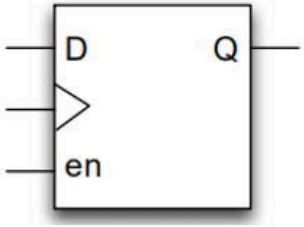
同步逻辑设计

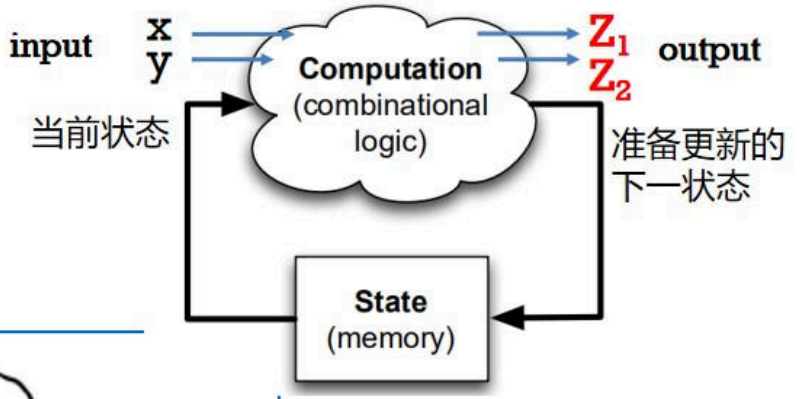


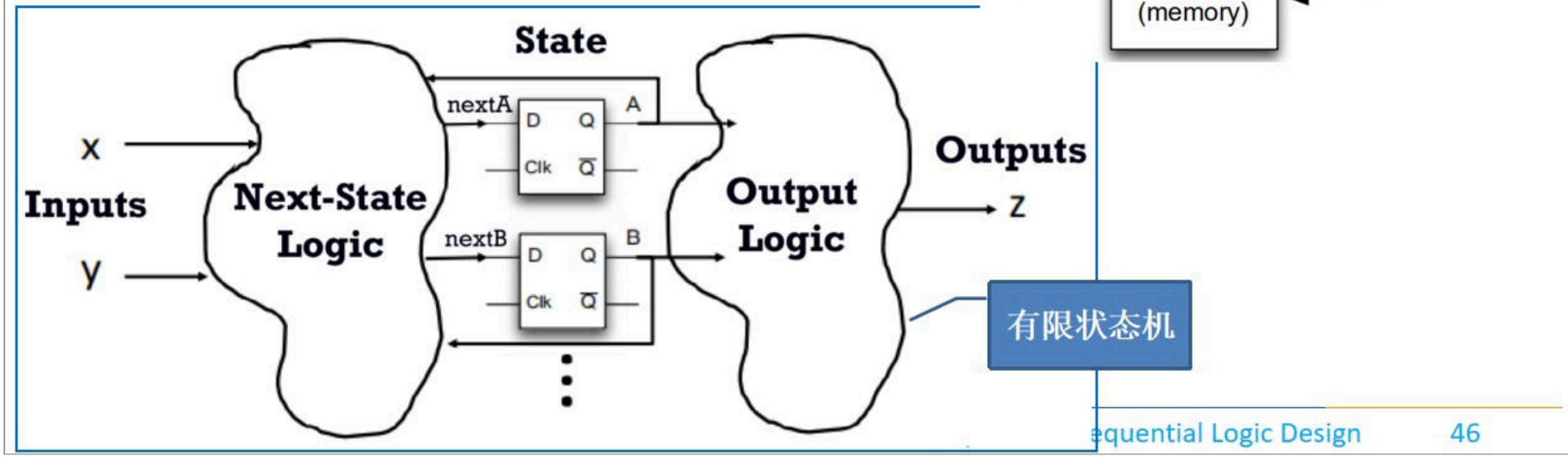


计算机最核心的工作就是，在“计算”和“更新状态”之间交替

同步逻辑设计 (慕课堂练习)



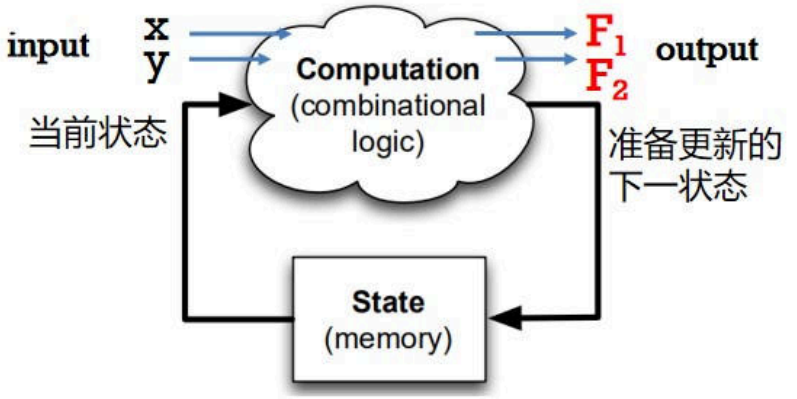


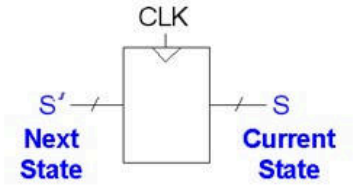


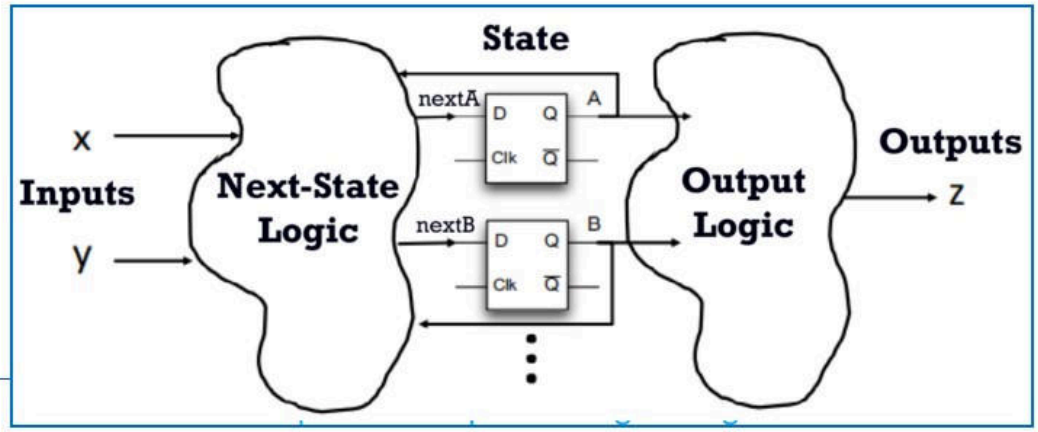
equential Logic Design 46

Finite State Machine (FSM)

- 有限状态机（FSM）由两部分构成：
  - 状态寄存器
    - 保存当前状态
    - 在时钟沿载入下一状态
  - 组合逻辑
    - 计算**下一状态**
    - 计算**输出**







NEU-SWC-A0801040040

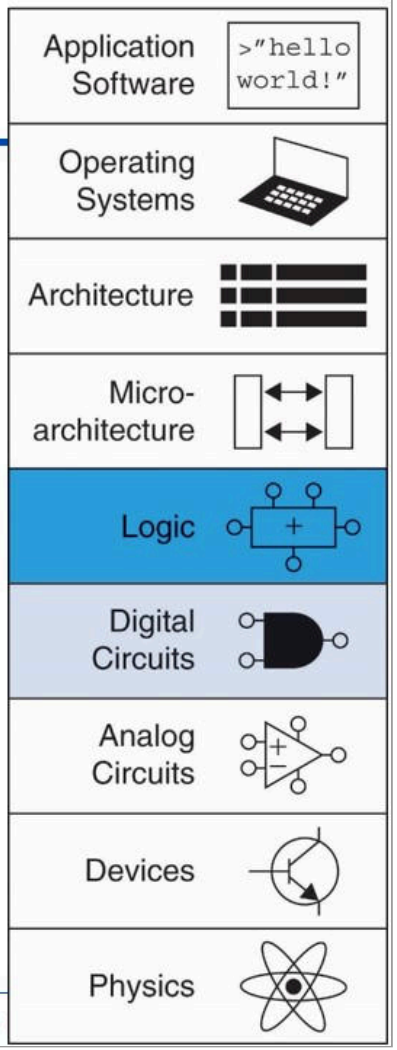
主观题 10分

在有限状态机（FSM）中，状态寄存器与组合逻辑分别承担什么功能？请结合交通灯控制器实例，说明二者如何协同工作以实现‘A路绿灯、B路红灯’到‘A路红灯、B路绿灯’的状态切换。

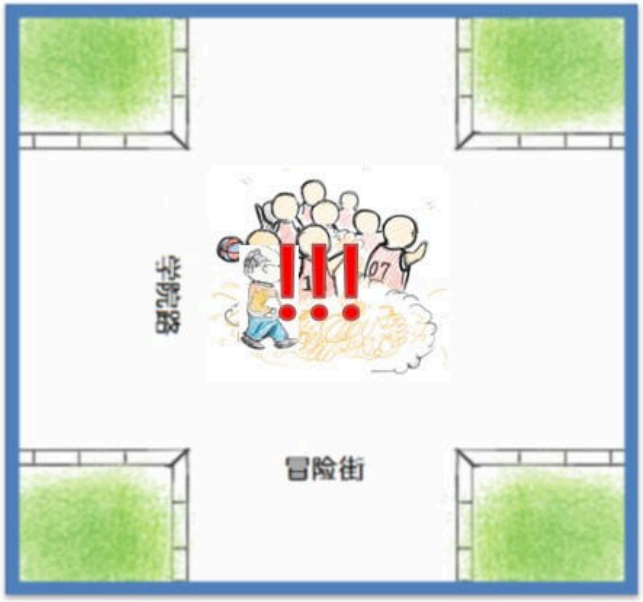
雨课堂 Rain Classroom

Chapter 3 :: Topics

- 引言
- 锁存器 (Latches) 和 触发器 (Flip-Flops)
- 寄存器 (Register) 和 寄存器文件 (Register File)
- 同步逻辑设计 (Synchronous Logic Design)
- **有限状态机 (Finite State Machines)**
- 时序逻辑的时序 (Timing of Sequential Logic)
- 并行 (Parallelism)



有限状态机 Finite State Machines

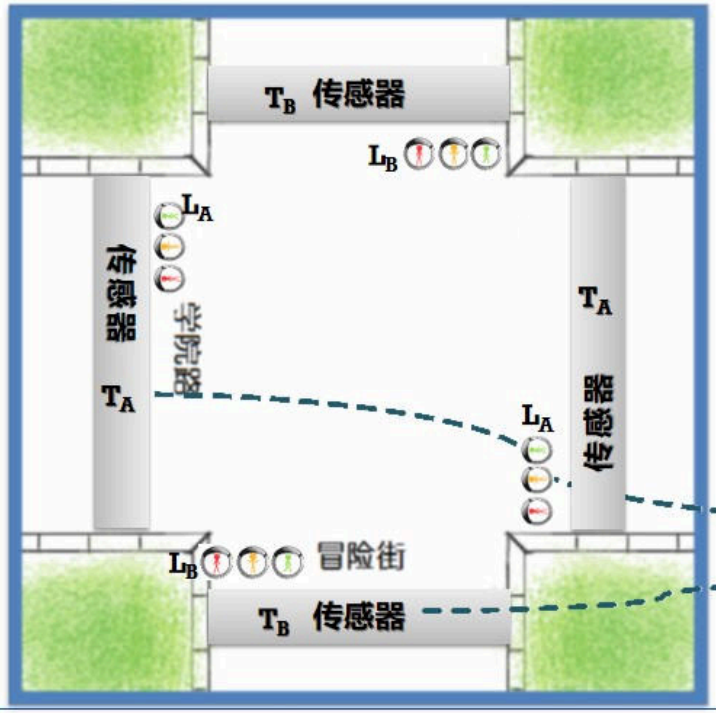


工程系学生徘徊在实验室和宿舍之间的学院路上，忙于阅读他们最爱的关于有限状态机的笔记而并不注意路上的情况。运动员们在运动场地和食堂之间的冒险街上推搡，来回传球，同样并没有在意周围的情况。因此，在两条道路交叉位置发生了极其严重的事故。

小明同学决定设计一个交通控制器解决此问题。

有限状态机

- 第一步：决定采用有限状态机方案，并做准备工作



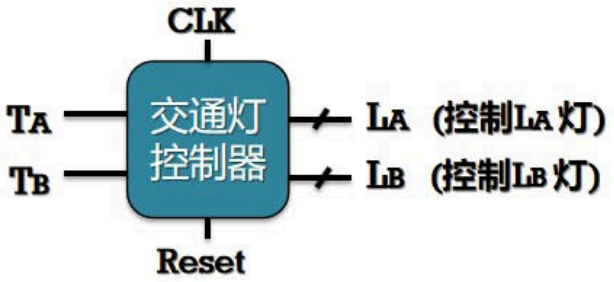
添加设备：



(L_A, L_B)

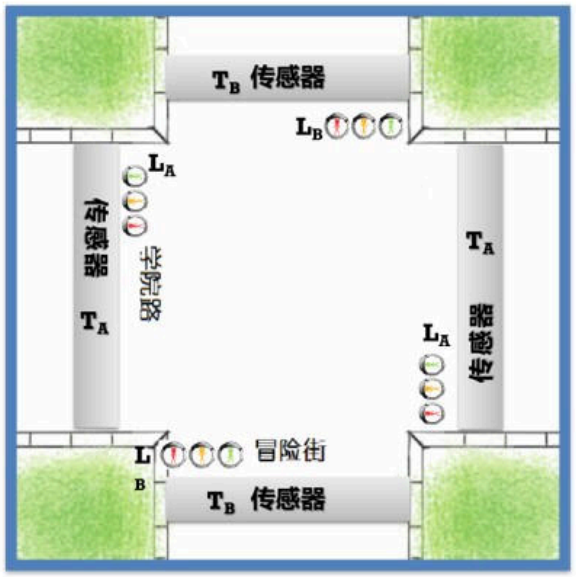
传感器 (T_A, T_B)

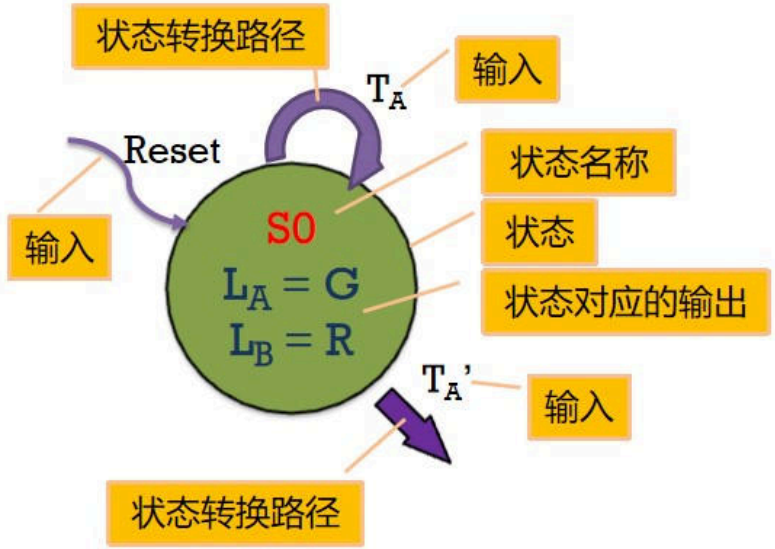
确定输入、输出：



有限状态机 （慕课堂练习）

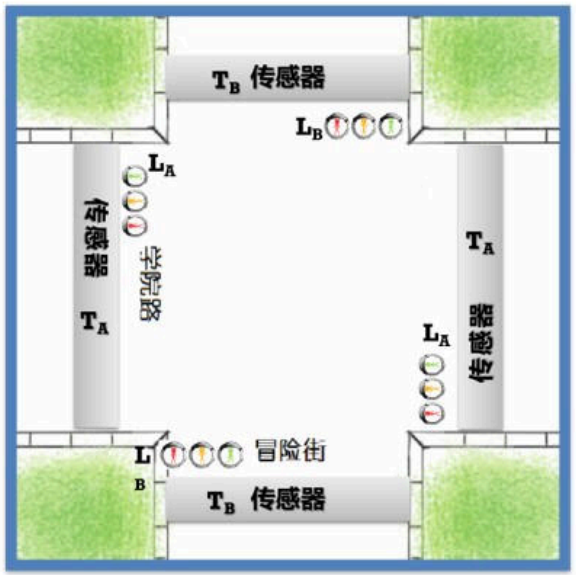
- 第二步：画出状态转换图
  - 状态图中，圆圈代表状态，圆弧代表两种状态之间的转换。

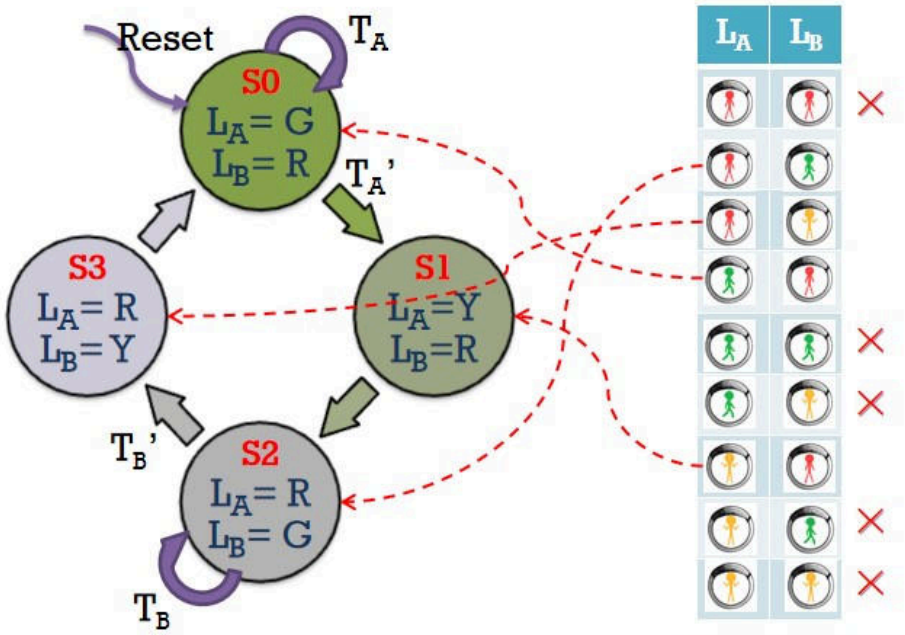




有限状态机 （慕课堂练习）

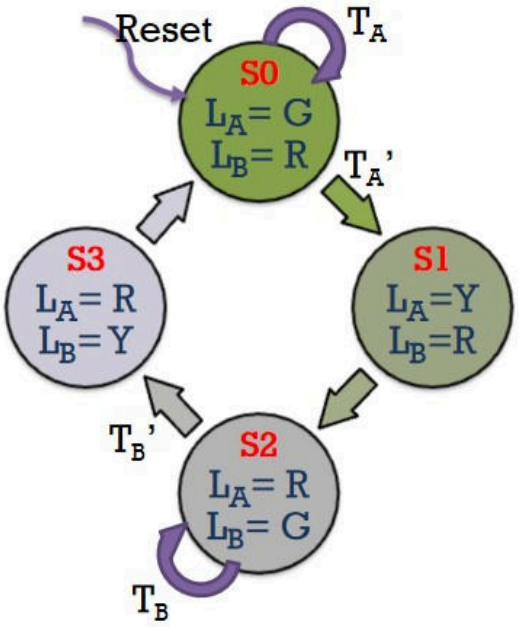
- 第二步：画出状态转换图
  - 状态图中，圆圈代表状态，圆弧代表两种状态之间的转换。





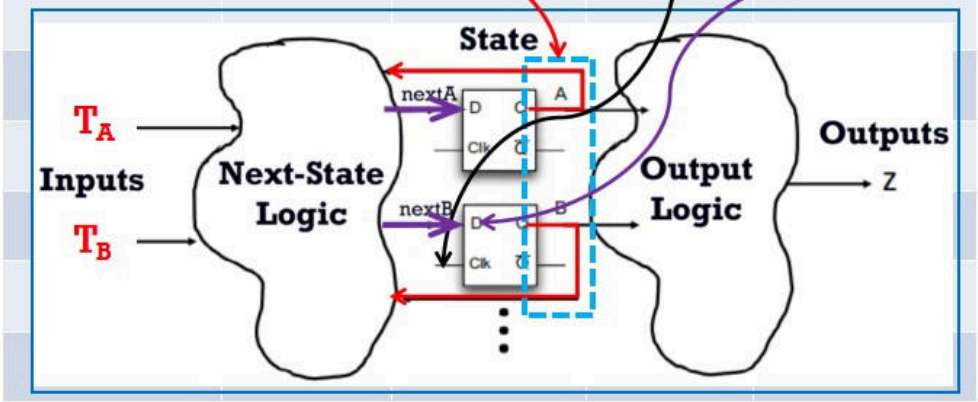
有限状态机

- 第三步：将状态转换图，转换为状态转换表



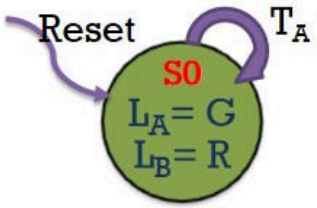
状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'



有限状态机

- 第三步：将状态转换图，转换为状态转移表

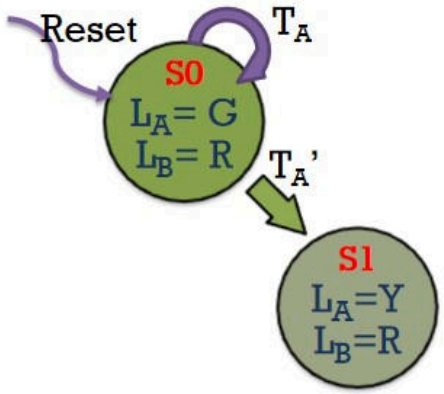


状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0

有限状态机

- 第三步：将状态转换图，转换为状态转移表



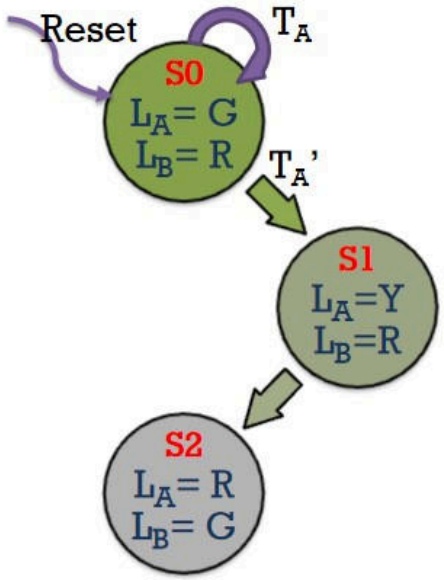
状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0
S0	0	×	↑	S1

雨课堂 Rain Classroom

有限状态机

- 第三步：将状态转换图，转换为状态转移表

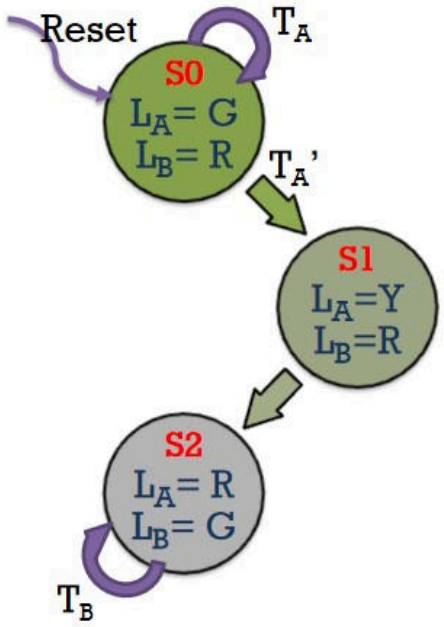


状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0
S0	0	×	↑	S1
S1	×	×	↑	S2

有限状态机

- 第三步：将状态转换图，转换为状态转移表

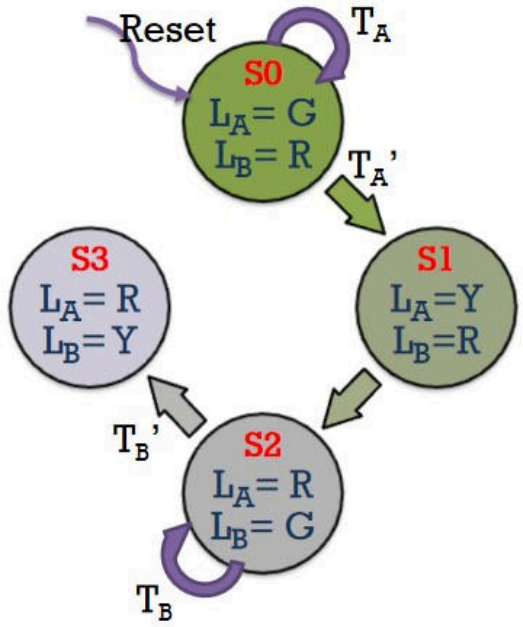


状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0
S0	0	×	↑	S1
S1	×	×	↑	S2
S2	×	1	↑	S2

有限状态机

- 第三步：将状态转换图，转换为状态转移表

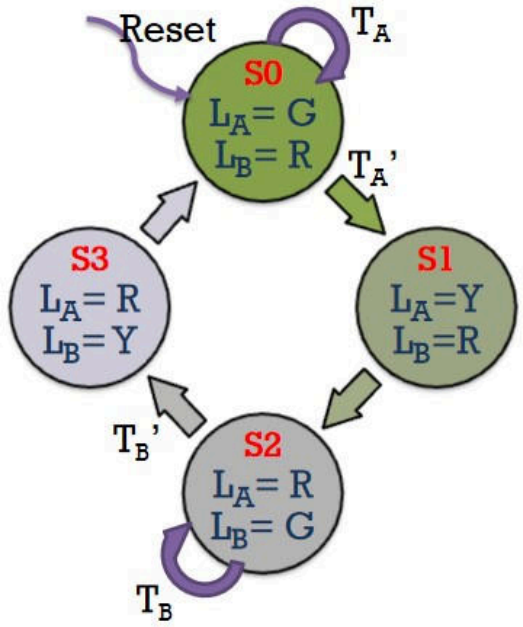


状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0
S0	0	×	↑	S1
S1	×	×	↑	S2
S2	×	1	↑	S2
S2	×	0	↑	S3

有限状态机

- 第三步：将状态转换图，转换为状态转移表



状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0
S0	0	×	↑	S1
S1	×	×	↑	S2
S2	×	1	↑	S2
S2	×	0	↑	S3
S3	×	×	↑	S0

有限状态机

- 此时的状态转移表，与真值表相似，但有不同
  - 输入S，是符号，而不是二进制
  - 输出S'，也是符号
  - 为了建立真实的电路，

    S 和 S' 都必须按照二进制编码

状态转移表

当前状态	输入			下一状态
S	T_A	T_B	CLK	S'
S0	1	×	↑	S0
S0	0	×	↑	S1
S1	×	×	↑	S2
S2	×	1	↑	S2
S2	×	0	↑	S3
S3	×	×	↑	S0

有限状态机

- 第四步：用二进制编码更新状态转换表

状态转移表

当前状态	输入			下一状态
S (S_1, S_0)	T_A	T_B	CLK	S' (S'_1, S'_0)
S0 (0,0)	1	×	↑	S0 (0,0)
S0 (0,0)	0	×	↑	S1 (0,1)
S1 (0,1)	×	×	↑	S2 (1,0)
S2 (1,0)	×	1	↑	S2 (1,0)
S2 (1,0)	×	0	↑	S3 (1,1)
S3 (1,1)	×	×	↑	S0 (0,0)

状态编码

状态	状态编码 $(S_{1:0})$
S0	0 0
S1	0 1
S2	1 0
S3	1 1

有限状态机

- 第五步：生成“下一状态”的布尔表达式

状态转移表

当前状态		输入			下一状态	
S_1	S_0	T_A	T_B	CLK	S'_1	S'_0
0	0	1	×	↑	0	0
0	0	0	×	↑	0	1
0	1	×	×	↑	1	0
1	0	×	1	↑	1	0
1	0	×	0	↑	1	1
1	1	×	×	↑	0	0

“下一状态”布尔表达式

$$S'_1 = \overline{S}_1 S_0 + S_1 \overline{S}_0 \overline{T}_B + S_1 \overline{S}_0 T_B$$

$$S'_0 = \overline{S}_1 \overline{S}_0 \overline{T}_A + S_1 \overline{S}_0 \overline{T}_B$$

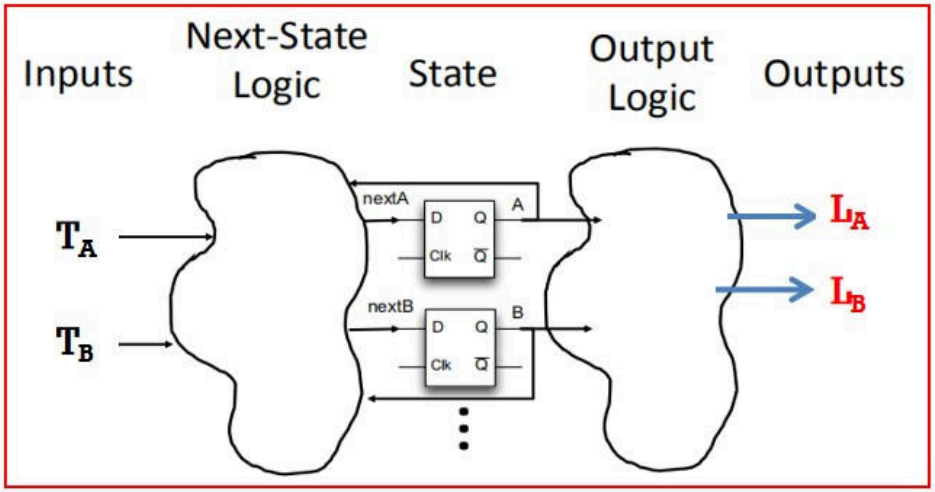
$$S'_1 = S_1 \oplus S_0$$

$$S'_0 = \overline{S}_1 \overline{S}_0 \overline{T}_A + S_1 \overline{S}_0 \overline{T}_B$$

雨课堂 Rain Classroom

有限状态机 (慕课堂练习)

- 第六步：针对每个状态指出的输出写出输出逻辑表



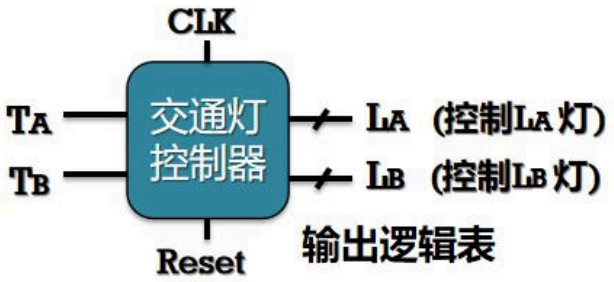


输出逻辑表

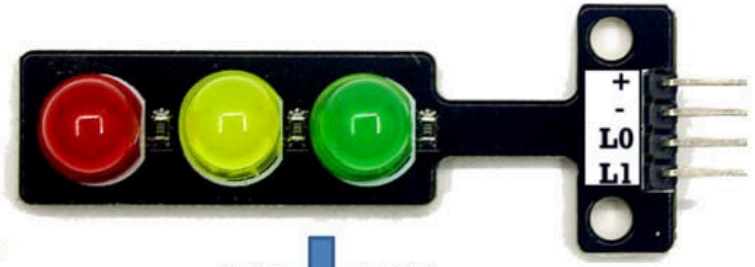
当前状态	输出	
S (S_1, S_0)	L_A	L_B
S0	G	R
S1	Y	R
S2	R	G
S3	R	Y

有限状态机

- 第七步：生成“输出”的布尔表达式



如何 对应？



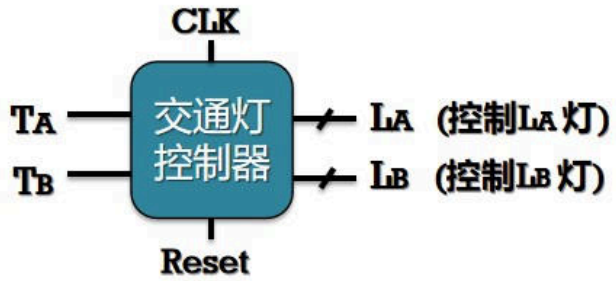
内部 逻辑

当前状态	输出	
S (S_1, S_0)	L_A	L_B
S0	G	R
S1	Y	R
S2	R	G
S3	R	Y

红绿灯模组输入	红绿灯状态		
L (L_1, L_0)	红R	黄Y	绿G
0, 0	0	0	1
0, 1	0	1	0
1, 0	1	0	0
1, 1	0	0	0

有限状态机

- 第七步：生成“输出”的布尔表达式



红绿灯模组输入	红绿灯状态		
L (L_1, L_0)	红R	黄Y	绿G
0, 0	0	0	1
0, 1	0	1	0
1, 0	1	0	0
1, 1	0	0	0

输出逻辑表

当前状态	输出	
S (S_1, S_0)	L_A	L_B
S0	G	R
S1	Y	R
S2	R	G
S3	R	Y

当前状态	输出	
S (S_1, S_0)	L_A(L_{A1}, L_{A0})	L_B(L_{B1}, L_{B0})
S0 (0,0)	G (0,0)	R (1,0)
S1 (0,1)	Y (0,1)	R (1,0)
S2 (1,0)	R (1,0)	G (0,0)
S3 (1,1)	R (1,0)	Y (0,1)

有限状态机

- 第七步：生成“输出”的布尔表达式

输出逻辑表

当前状态	输出	
S (S_1, S_0)	L_A(L_{A1}, L_{A0})	L_B(L_{B1}, L_{B0})
S0 (0,0)	G (0,0)	R (1,0)
S1 (0,1)	Y (0,1)	R (1,0)
S2 (1,0)	R (1,0)	G (0,0)
S3 (1,1)	R (1,0)	Y (0,1)

当前状态		输出			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

“输出”的布尔表达式

$$L_{A1} = S_1$$

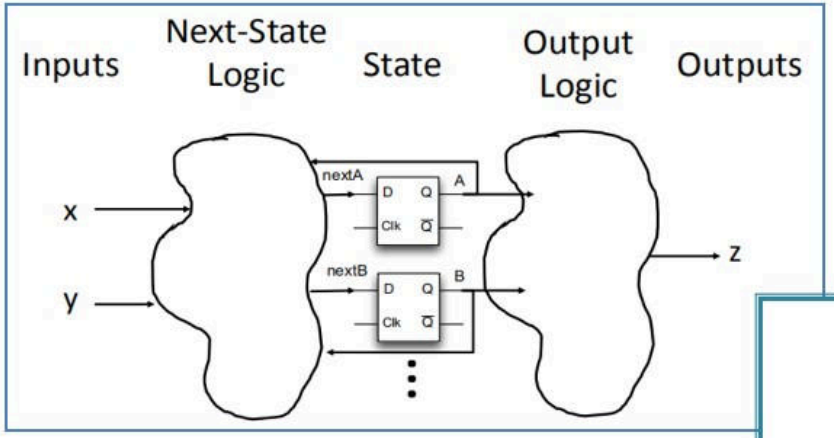
$$L_{A0} = \overline{S}_1 S_0$$

$$L_{B1} = \overline{S}_1$$

$$L_{B0} = S_1 S_0$$

有限状态机

- 第八步：根据上述布尔表达式，绘制有限状态机的电路图

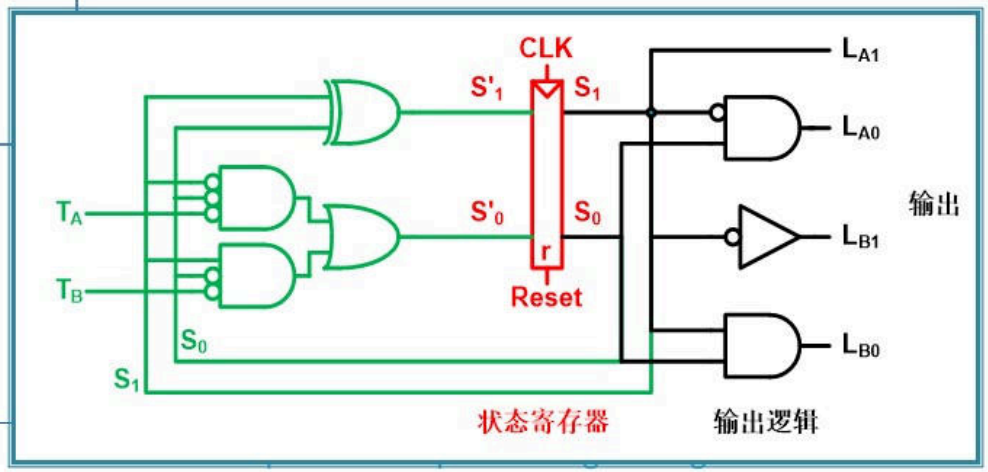


“输出”的布尔表达式

$$L_{A1} = S_1 \qquad L_{B1} = \overline{S}_1$$
$$L_{A0} = \overline{S}_1 S_0 \qquad L_{B0} = S_1 S_0$$

“下一状态”布尔表达式

$$S'_1 = S_1 \oplus S_0$$
$$S'_0 = \overline{S}_1 \overline{S}_0 \overline{T}_A + S_1 \overline{S}_0 \overline{T}_B$$



NEU-SWC-A0801040040

有限状态机设计流程

- 有限状态机是根据给定规范设计同步时序电路的强有力的系统化方法。

  设计有限状态机的步骤如下：
  - 确定输入和输出
  - 画状态转换图
  - 写出状态转换表
  - 选择状态编码——这将影响硬件设计
  - 对于 Moore 型有限状态机，根据选择的状态编码形式，重新写出二进制的状态转换表和二进制的输出表
  - 为“下一个状态”和“输出”写出布尔表达式
  - 画出电路草图。

雨课堂 Rain Classroom

单选题 1分

在有限状态机的状态转换图中，以下哪一项正确描述了圆弧的含义？

A 表示电路的时钟周期

B 表示状态内部的自循环

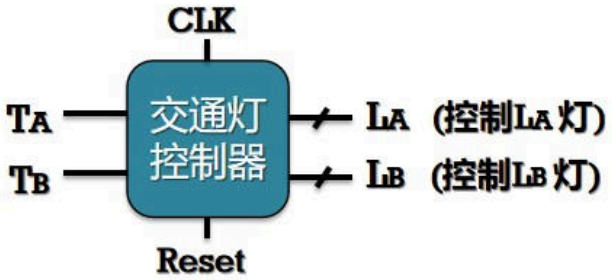
C 表示两种状态之间的转换，由输入信号触发

D 表示输出信号的波形变化

雨课堂 Rain Classroom

有限状态机

- 进一步思考：





输出逻辑表

当前状态	输出	
S (S_1, S_0)	L_A	L_B
S0	G	R
S1	Y	R
S2	R	G
S3	R	Y

假设红绿灯模组的接口如上图

该如何数字化输出逻辑表?

理解二进制编码和独热编码的区别

雨课堂 Rain Classroom

有限状态机的状态编码

- 二进制 Binary 编码方式：
  - 例如，对于四种状态，二进制编码为：00, 01, 10, 11
- 独热 One-hot 编码方式：
  - 一个状态占编码的一位
  - 众多状态中，某时刻只有一个状态位为 HIGH
  - 例如，对于四种 状态，独热编码为：0001, 0010, 0100, 1000
  - 独热编码方式需要更多的触发器来存储状态位
  - 但常常下一状态逻辑和输出逻辑变得简单

有限状态机

状态转移表

当前状态	输入			下一状态
S (S_1, S_0)	T_A	T_B	CLK	S' (S'_1, S'_0)
S0 (0,0)	1	×	↑	S0 (0,0)
S0 (0,0)	0	×	↑	S1 (0,1)
S1 (0,1)	×	×	↑	S2 (1,0)
S2 (1,0)	×	1	↑	S2 (1,0)
S2 (1,0)	×	0	↑	S3 (1,1)
S3 (1,1)	×	×	↑	S0 (0,0)

当前状态	输入			下一状态
S (S_3, S_2, S_1, S_0)	T_A	T_B	CLK	S'(S'_1, S'_1, S'_1, S'_0)
S0 (0,0,0,1)	1	×	↑	S0 (0,0,0,1)
S0 (0,0,0,1)	0	×	↑	S1 (0,0,1,0)
S1 (0,0,1,0)	×	×	↑	S2 (0,1,0,0)
S2 (0,1,0,0)	×	1	↑	S2 (0,1,0,0)
S2 (0,1,0,0)	×	0	↑	S3 (1,0,0,0)
S3 (1,0,0,0)	×	×	↑	S0 (0,0,0,1)

状态S的二进制编码

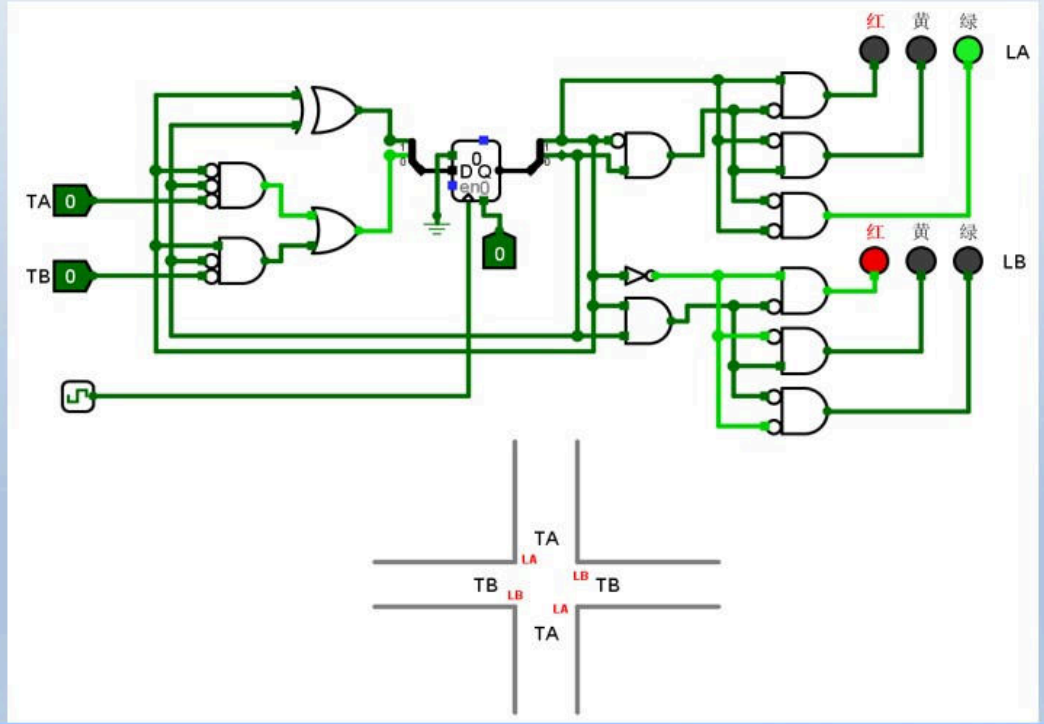
状态	状态编码($S_{1:0}$)
S0	0 0
S1	0 1
S2	1 0
S3	1 1

状态S的独热编码

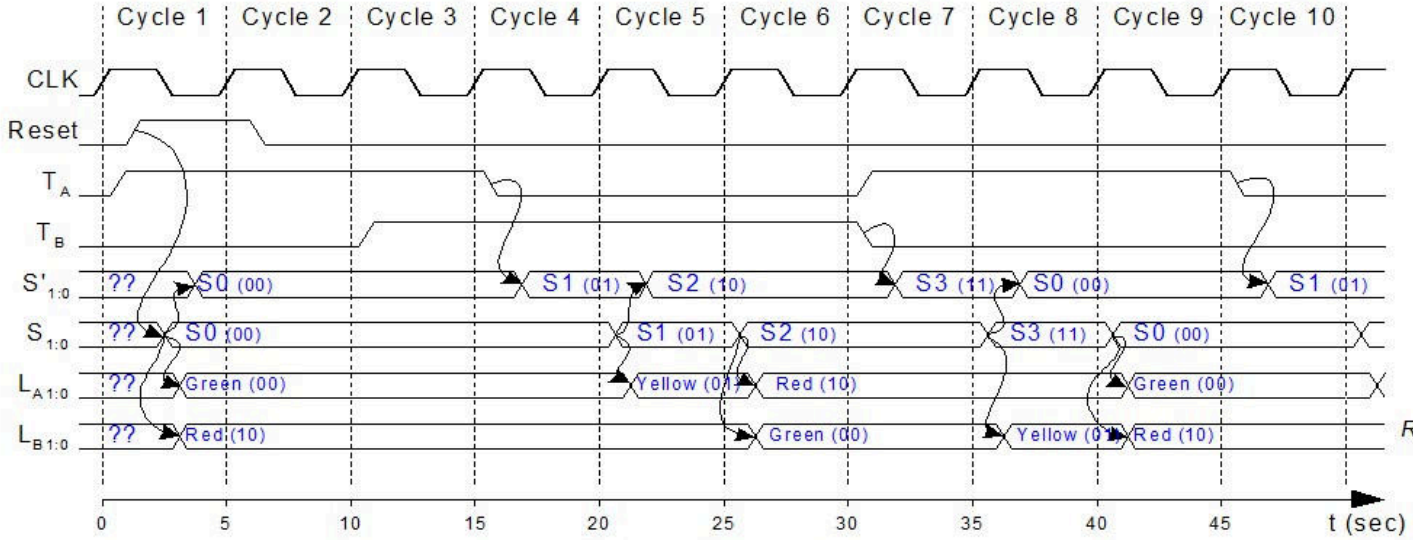
状态	状态编码($S_{3:0}$)
S0	0 0 0 1
S1	0 0 1 0
S2	0 1 0 0
S3	1 0 0 0

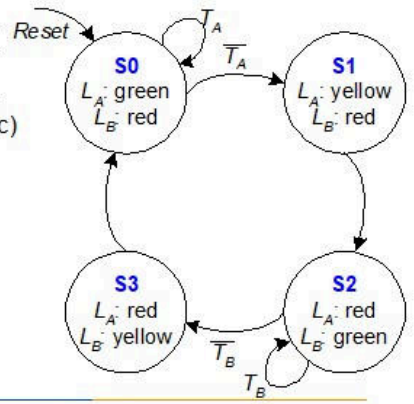
课后作业1

- 设计交通控制器，并使用Logisim实现



有限状态机的时序图 (课后自学)





多选题 1分

关于有限状态机的状态编码方式，以下哪些说法是正确的?

A 对于4个状态，二进制编码需要2位，独热编码需要4位

B 独热编码下，任意时刻仅有一个状态位为1（HIGH），其余为0

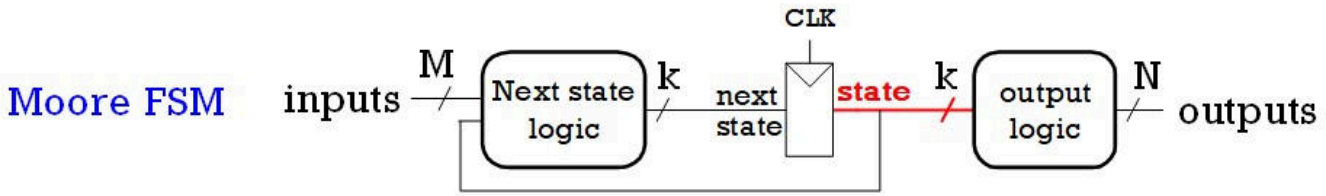
C 二进制编码通常使下一状态逻辑更简洁，而独热编码使输出逻辑更复杂

D 独热编码虽增加触发器数量，但可避免状态解码电路，简化组合逻辑

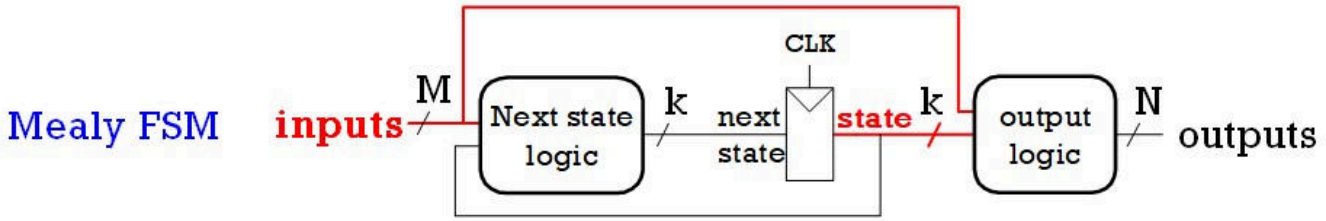
雨课堂 Rain Classroom

有限状态机的类型

- 有限状态机分两种类型，依据它们的输出逻辑:
  - **Moore FSM**: 输出逻辑只依赖于当前的状态

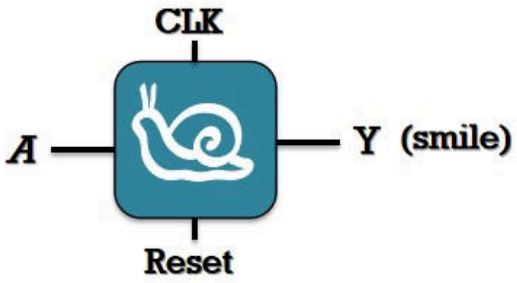


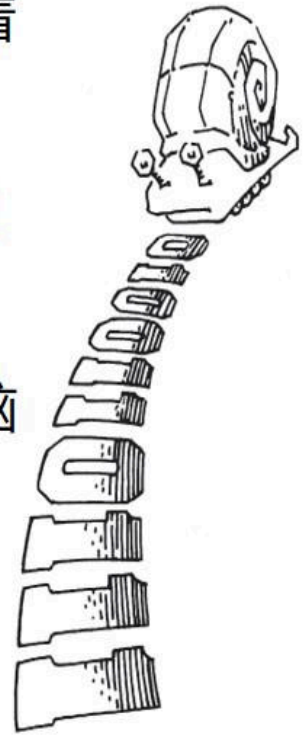
  - **Mealy FSM**: 输出逻辑依赖于当前的状态和输入



Moore vs. Mealy 型有点状态机

- Alyssa有一个带有限状态机大脑的机器宠物蜗牛。蜗牛沿着纸带爬行。纸带上包含1和0的序列。在每一个时钟周期，蜗牛爬到下一位。如果最后经过的2位是01，蜗牛就会高兴的笑起来。
- 分别设计一个摩尔型和米勒型的有限状态机作为蜗牛的大脑，判断蜗牛笑不笑。



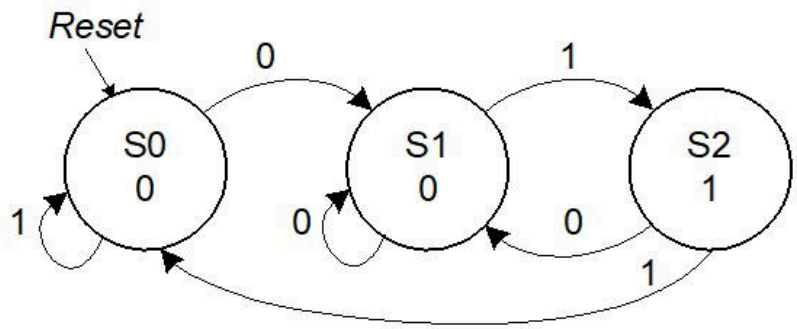


摩尔型有限状态机的状态转移图

- 由于Moore型有限状态机的每个输出只取决于系统的当前状态(current state)，所以就将输出(output)标记在圆圈内（每个圆圈代表一个状态）

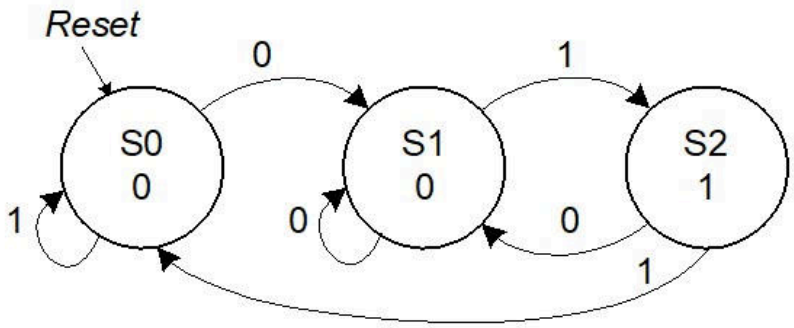
  - S0：一直都是读到“1”的状态
  - S1：一直都是读到“0”的状态
  - S2：读到“0”又读到“1”的状态

Moore FSM



摩尔型有限状态机的状态转移表

Moore FSM



状态转移表

当前状态	输入		下一状态
S	A	CLK	S'
S0	0	↑	S1
S0	1	↑	S0
S1	0	↑	S1
S1	1	↑	S2
S2	0	↑	S1
S2	1	↑	S0

摩尔型有限状态机的状态转移表

状态编码

状态	状态编码 ($S_{1:0}$)
S0	0 0
S1	0 1
S2	1 0



状态转移表

当前状态	输入		下一状态
S ($S_{1:0}$)	A	CLK	S' ($S'_{1:0}$)
S0 (0,0)	0	↑	S1 (0,1)
S0 (0,0)	1	↑	S0 (0,0)
S1 (0,1)	0	↑	S1 (0,1)
S1 (0,1)	1	↑	S2 (1,0)
S2 (1,0)	0	↑	S1 (0,1)
S2 (1,0)	1	↑	S0 (0,0)

摩尔型有限状态机的状态转移表

状态转移表

当前状态	输入		下一状态
S ($S_{1:0}$)	A	CLK	S' ($S'_{1:0}$)
S0 (0,0)	0	↑	S1 (0,1)
S0 (0,0)	1	↑	S0 (0,0)
S1 (0,1)	0	↑	S1 (0,1)
S1 (0,1)	1	↑	S2 (1,0)
S2 (1,0)	0	↑	S1 (0,1)
S2 (1,0)	1	↑	S0 (0,0)

S'_1

A \ S_1S_0	00	01	11	10
0	0	0	×	0
1	0	1	×	0

$$S_1' = S_0 A$$

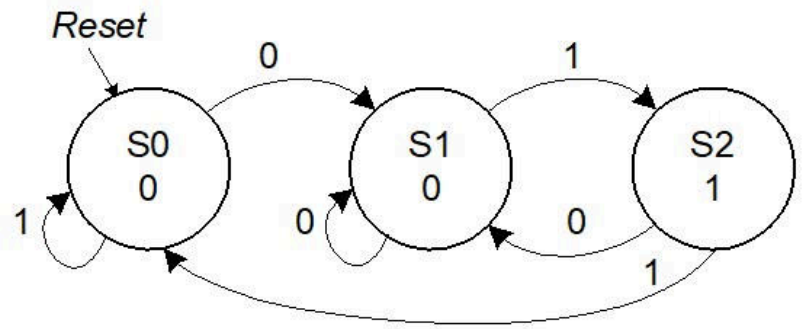
S'_0

A \ S_1S_0	00	01	11	10
0	1	1	X	1
1	0	0	X	0

$$S_0' = A'$$

摩尔型状态机的输出表

Moore FSM



输出逻辑表

当前状态	输出
S (S_1, S_0)	**Y**
S0 (0,0)	0
S1 (0,1)	0
S2 (1,0)	1

Y (S_1 \ S_0)	0	1
0	0	0
1	1	×

$$Y = S_1$$

雨课堂 Rain Classroom

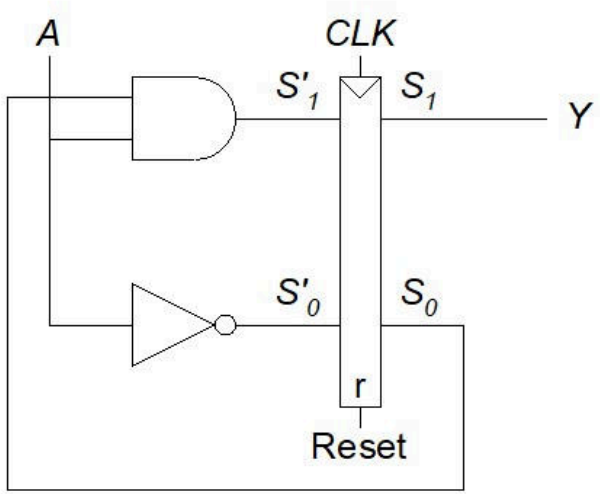
摩尔型状态机的电路图

下一状态逻辑：

$$S_1' = S_0 A$$
$$S_0' = A'$$

输出逻辑：

$$Y = S_1$$

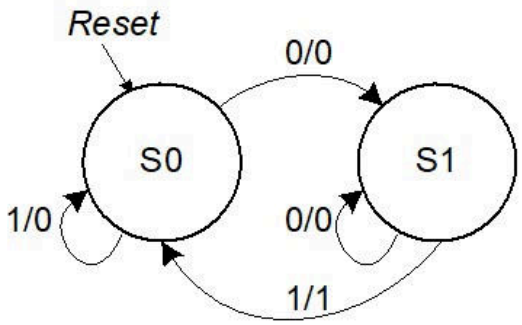


Mealy型有限状态机的状态转移图

- 由于Mealy型有限状态机的每个输出取决于输入和当前系统状态，所以就将输出标记在了圆弧上，而不是圆圈内（圆弧上标注为A/Y, A为引起状态转换的输入，Y为输出）

  - S0：当前读到“1”的状态
  - S1：当前读到“0”的状态

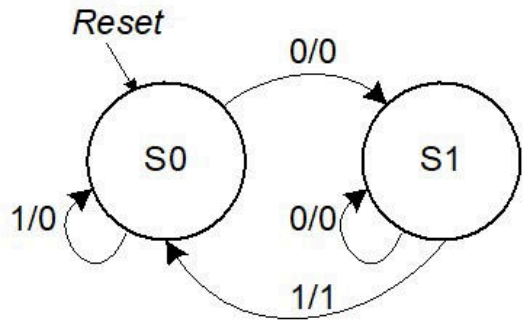
Mealy FSM



Mealy FSM: arcs indicate input/output

Mealy型有限状态机的状态转移表 & 输出表

Mealy FSM



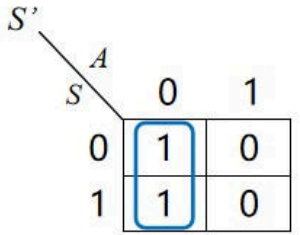
状态转移表 & 输出表

当前状态	输入		下一状态	输出
S	A	CLK	S'	Y
S0	0	↑	S1	0
S0	1	↑	S0	0
S1	0	↑	S1	0
S1	1	↑	S0	1

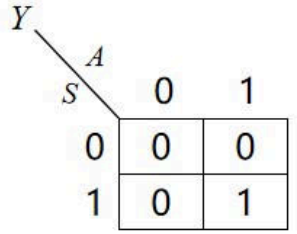
Mealy型有限状态机的状态转移表 & 输出表

状态转移表 & 输出表

当前状态	输入		下一状态	输出
S	A	CLK	S'	Y
S0 (0)	0	↑	S1 (1)	0
S0 (0)	1	↑	S0 (0)	0
S1 (1)	0	↑	S1 (1)	0
S1 (1)	1	↑	S0 (0)	1



$S' = A'$



$Y = SA$

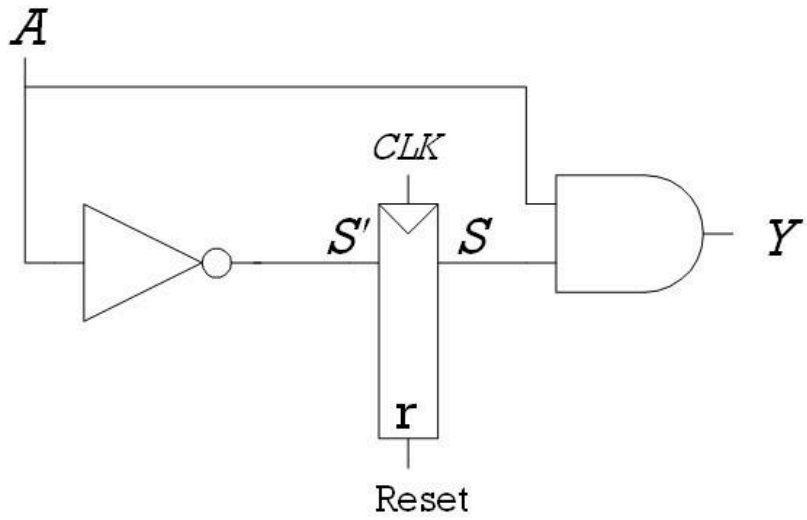
雨课堂 Rain Classroom

Mealy型有限状态机的电路图

- 下一状态逻辑

S' \ A,S	0	1
0	1	0
1	1	0

$S' = A'$

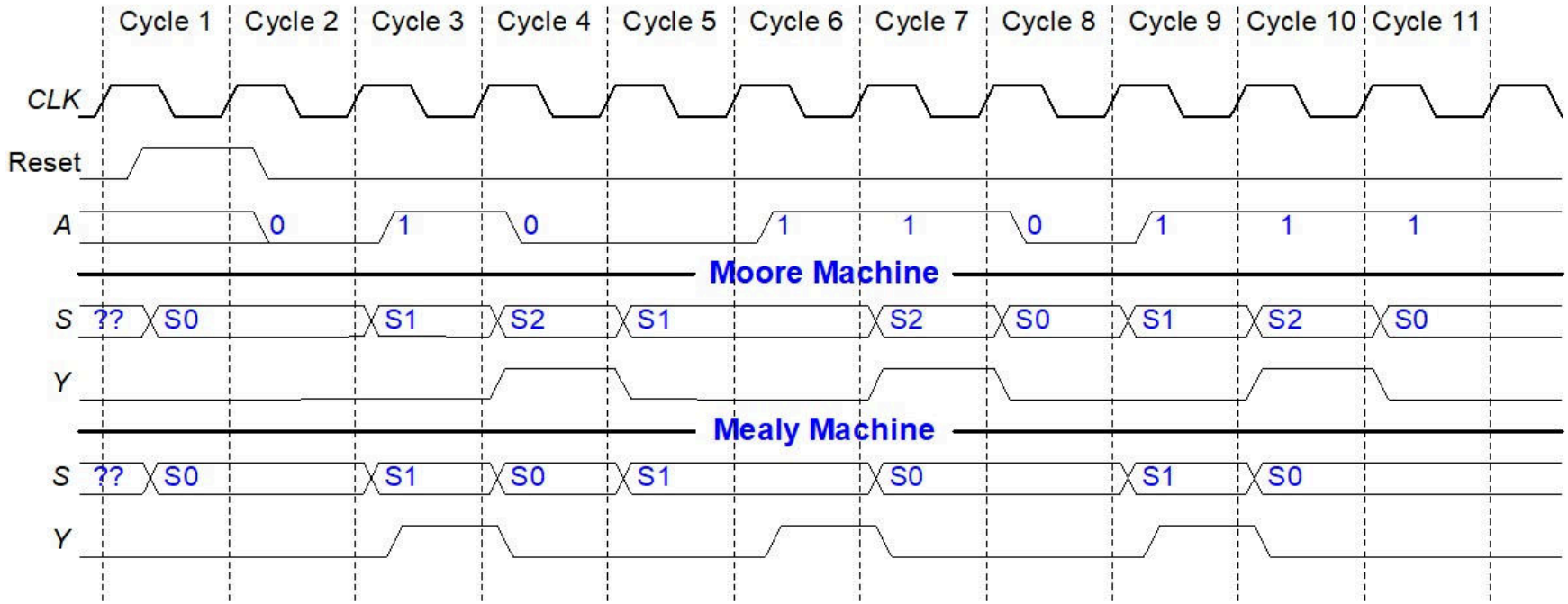


- 输出逻辑

Y \ A,S	0	1
0	0	0
1	0	1

$Y = SA$

摩尔型状态机和米勒型状态机的时序图 （课后自学）



FSM Design Procedure

- 有限状态机是根据给定规范设计同步时序电路的强有力的系统化方法。设计有限状态机的步骤如下：
  - 确定输入和输出
  - 画状态转换图
  - 选择状态编码——这将影响硬件设计
  - **对于摩尔型状态机**：重写二进制的状态转移表和输出表
  - **对于米勒型状态机**：重新写出二进制的状态转移表和输出表
  - 为“下一个状态”和“输出”写出布尔表达式
  - 画出电路草图。

单选题 1分

Mealy型有限状态机与Moore型有限状态机最根本的区别在于：

A Mealy型使用D触发器，Moore型使用JK触发器

B Mealy型的状态转移速度更快

C Mealy型的输出仅取决于当前状态，Moore型的输出取决于当前状态和当前输入

D Mealy型的输出取决于当前状态和当前输入，Moore型的输出仅取决于当前状态

雨课堂 Rain Classroom

第九堂课结束了

有问题同学们可以提问讨论

第十堂：小班讨论课

有限状态机的设计

- 有一台自动售糖机，可以销售 15-cents（美分）的糖果
- 它可以接受两种硬币的投币
  - 镍币 (5-cents)
  - 一角硬币 (10-cents)
- 如果余额 ≥ 15 美分，则售出糖果（自动吐出糖果）
- 当顾客投币金额多了（如投了2个10美分）
  - 自动售糖机不退回零钱，但会为未来的交易保留余额



雨课堂 Rain Classroom

有限状态机的设计

- 步骤 1：确认输入，输出和状态
  - Input：（投入的硬币）
  - Output：（出、不出糖果）
  - State：（余额情况）

有限状态机的设计

- 步骤 2：绘制状态转移图

雨课堂 Rain Classroom

有限状态机的设计

- 步骤 3：写出状态转移表

current state	input		next current
S	d	n	S*

雨课堂 Rain Classroom

有限状态机的设计

- 步骤 4：选择状态编码方式

current state	input		next current
S	d	n	S*

Binary encoding(5 states: How many bits?)

state	Binary encoding

One-Hot encoding

state	One-Hot encoding

雨课堂 Rain Classroom

有限状态机的设计

- 步骤5：用选定的状态编码方式重写状态转移表

current state	input		next current
S (s_2, s_1, s_0)	d	n	S*(s^*_2, s^*_1, s^*_0)

Write output table (For Moore)

current state	output c

Binary encoding

state	Binary encoding

雨课堂 Rain Classroom

有限状态机的设计

- 步骤6：对于每一个下一状态逻辑和输出逻辑写出布尔表达式

current state	input		next current
S (s_2, s_1, s_0)	d	n	S*(s^*_2, s^*_1, s^*_0)

s^*_2 =

s^*_1=

$s_2s_1s_0$ / dn	000	001	011	010	110	111	101	100
00				1	x	x	x	
01		1		1	x	x	x	1
11	x	x	x	x	x	x	x	x
10	1	1	1		x	x	x	1

$s^*_0 = s_2's_0'n + s_1's_0n' + s_1n + s_2n'$

$s_2s_1s_0$ / dn	000	001	011	010	110	111	101	100
00		1			x	x	x	1
01	1		1	1	x	x	x	
11	x	x	x	x	x	x	x	x
10		1			x	x	x	1

雨课堂 Rain Classroom

有限状态机的设计

- 步骤6：对于每一个下一状态逻辑和输出逻辑写出布尔表达式

current state $S(s_2, s_1, s_0)$	output c

c =

s_2s_1 / s_0	00	01	11	10
0			x	1
1		1	x	x

雨课堂 Rain Classroom

有限状态机的设计

- 步骤7：绘制逻辑电路图
- 下一状态逻辑
  - s*2 =
  - s*1=
  - s*0=
- 输出逻辑
  - c =

有限状态机的设计

- 步骤5：用选定的状态编码方式重写状态转移表

current state	input		next current
$S(s_{4:0})$	d	n	$S^*(s^*_{4:0})$

Write output table (For Moore)

current state	output c

One-Hot encoding

state	One-Hot encoding

有限状态机的设计

- 步骤6：对于每一个下一状态逻辑和输出逻辑写出布尔表达式

current state	input		next current
$S(s_{4:0})$	d	n	$S^*(s^*_{4:0})$

Next State Logic

$s^*_4 =$

$s^*_3 =$

$s^*_2 =$

$s^*_1 =$

$s^*_0 =$

Output Logic

$c =$

current state	output c

有限状态机的设计

- 步骤7：绘制逻辑电路图
- 下一状态逻辑
  - s*4 =
  - s*3=
  - s*2=
  - s*1=
  - s*0=
- 输出逻辑
  - c =

有限状态机的设计

- 对于 Moore 型有限状态机:
  - 用选定的状态编码方式重写状态转移表
  - 写出输出表

雨课堂 Rain Classroom

课后作业2

- 设计自动售糖机，并使用Logisim实现
- 要求：
  - 以小组为单位进行讨论
  - 针对Moore型、Mealy型、二进制的状态编码方式、独热编码方式，给出不同的设计方案，并讨论不同方案的优劣。
  - 每名同学负责一个方案的Logisim实现。

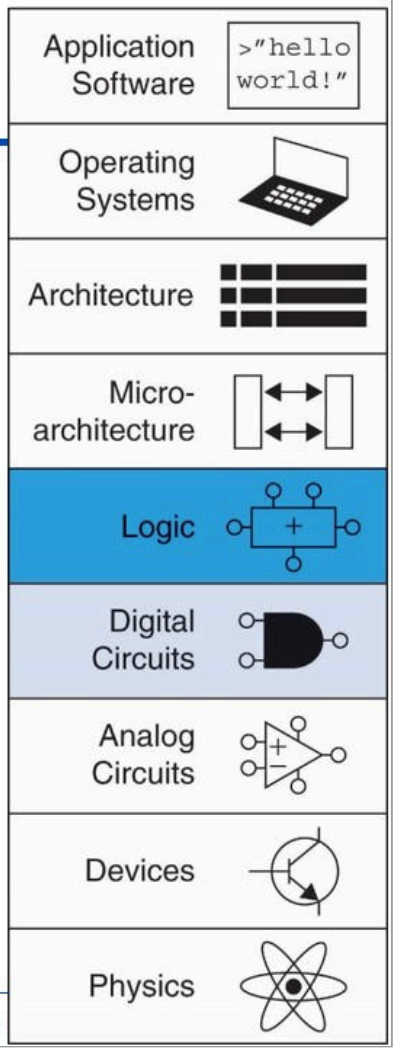
第十堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

Chapter 3 :: Topics

- 引言
- 锁存器 (Latches) 和 触发器 (Flip-Flops)
- 寄存器 (Register) 和 寄存器文件 (Register File)
- 同步逻辑设计 (Synchronous Logic Design)
- 有限状态机 (Finite State Machines)
- **时序逻辑的时序 (Timing of Sequential Logic)**
- 并行 (Parallelism)



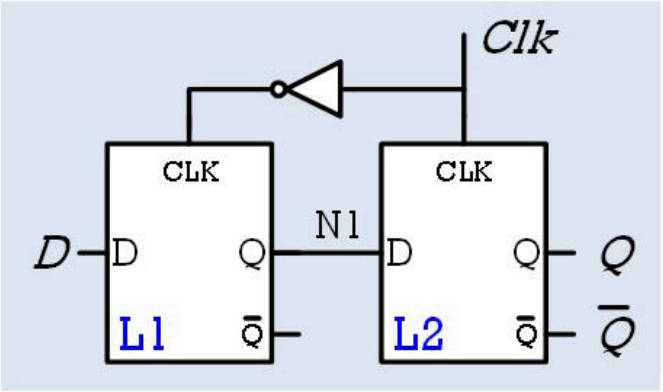
时序 Timing

- CPU为什么不能随便超频



时序 Timing

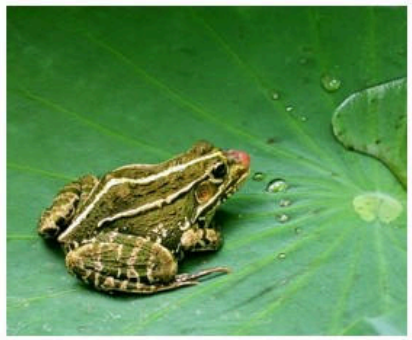
- 触发器在时钟**边沿采样**输入 *D*
- *D* 信号在被采样的时必须是稳定的



- *Clk* 低电平期间 *D*→N1，*Clk* 跳变到高电平 N1→*Q*
- 当在时钟上升沿的时候，如果 *D* 是稳定的0或者1，那么采样结果很清晰，如果在时钟上升沿的时候 *D* 不稳定，则不好说了。

时序

- 这个情景就像用照相机拍青蛙。照相机在孔径时间取像，如果在孔径时间内青蛙不动，则取像清晰。
- 触发器在采样的时钟边沿，输入D必须状态稳定，否则采样值不确切

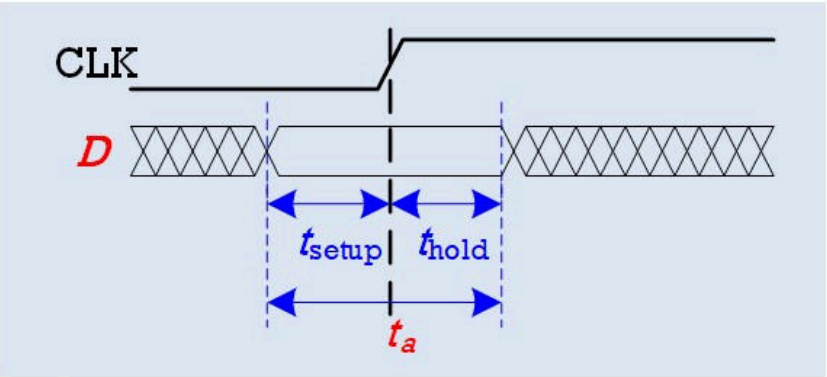


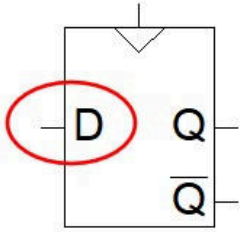




输入的时序约束

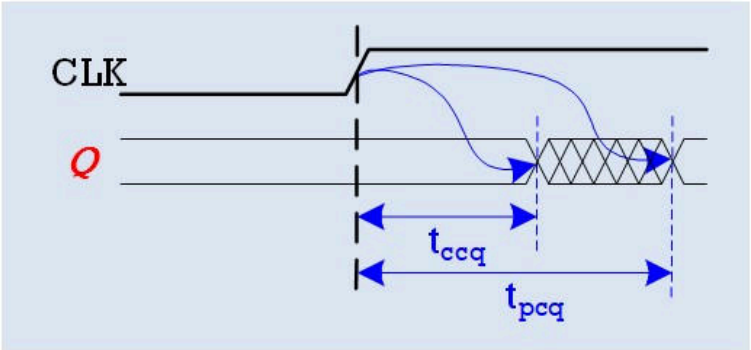
- 孔径时间（Aperture time）：t_a = 输入数据在时钟沿前后必须保持稳定的时间
  - $t_a = t_{setup} + t_{hold}$
- 建立时间（Setup time）： t_{setup} = 在时钟边沿到来之前，数据必须已经稳定（即不发生变化）的时间长度
- 稳定保持时间（Hold time）：t_{hold} = 时钟边沿之后数据还必须保持稳定的时间长度

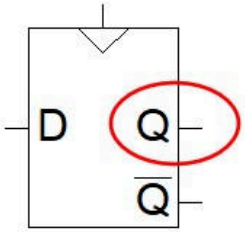




输出的时序约束

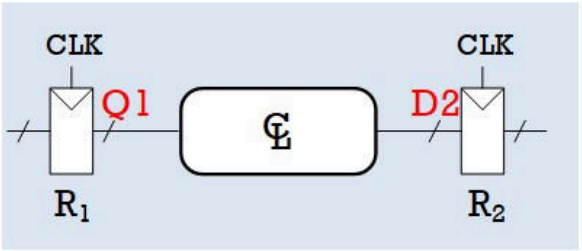
- 触发器的输出Q从**前一状态**到**当前状态**的变化过程中，经历了最小延迟和传输延迟
- 最小延迟（Contamination delay）：t_{ccq} = 输出 Q 在时钟边沿之后的最小延迟 t_{ccq} 之后开始不稳定了（即开始改变）
- 传输延迟（Propagation delay）：t_{pcq} = 输出 Q 在时钟沿之后的传输延迟 t_{pcq} 之后到达稳定值（即不再变化了）



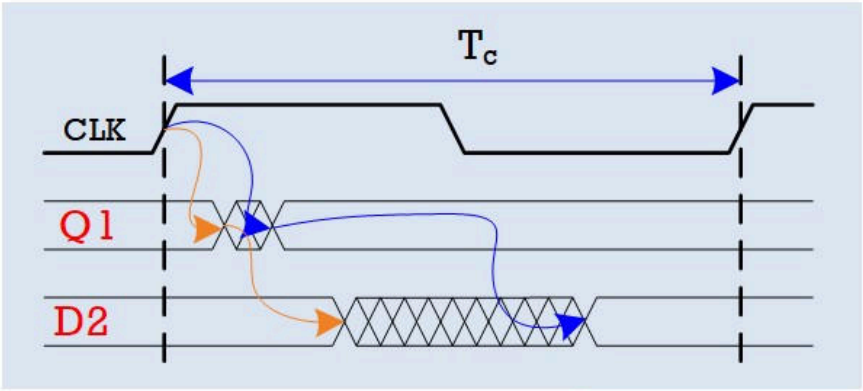


动态约束

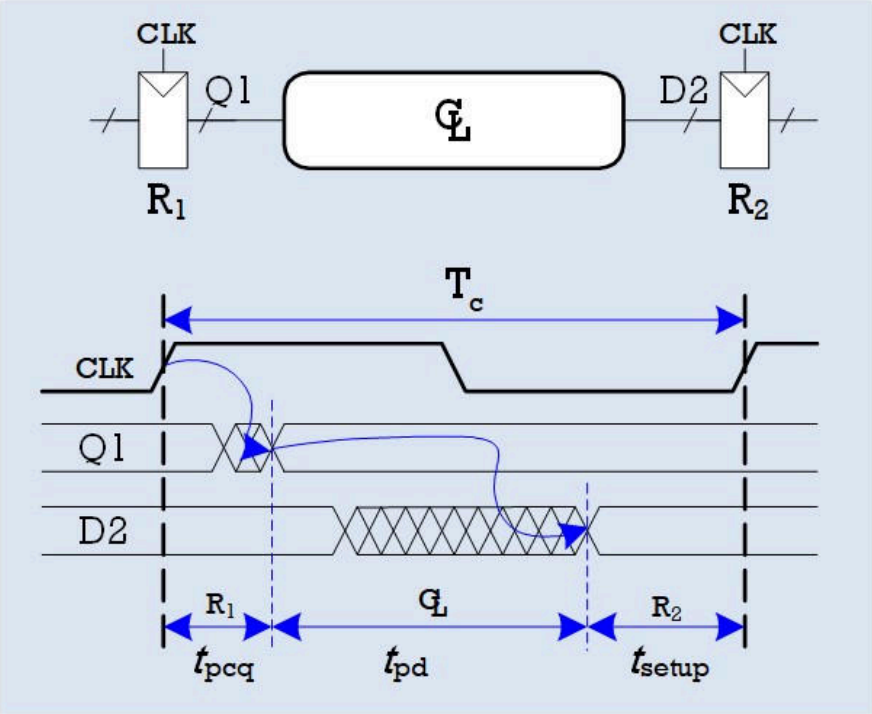
- 在同步时序电路中，不可能只有一个寄存器，我们抽象出这样的一条通用路径（如右图），希望**计算出它的时钟周期**
- 在时钟上升沿，寄存器R1产生输出Q1,Q1进入组合逻辑电路产生D2输出，D2又作为寄存器R2的输入



- 每一个输出信号，都在输入信号发生改变的最小延迟开始改变（橘色箭头指示）
- 在输入信号稳定之后的传输延迟时间内稳定到最终的值。

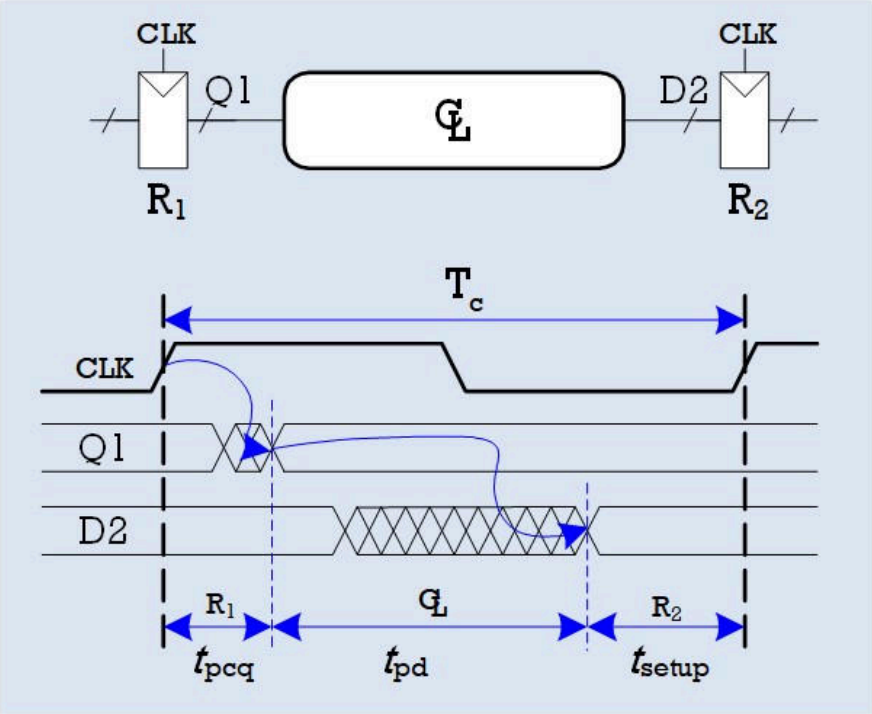


建立时间的时序约束



$T_c \geq$

建立时间的时序约束

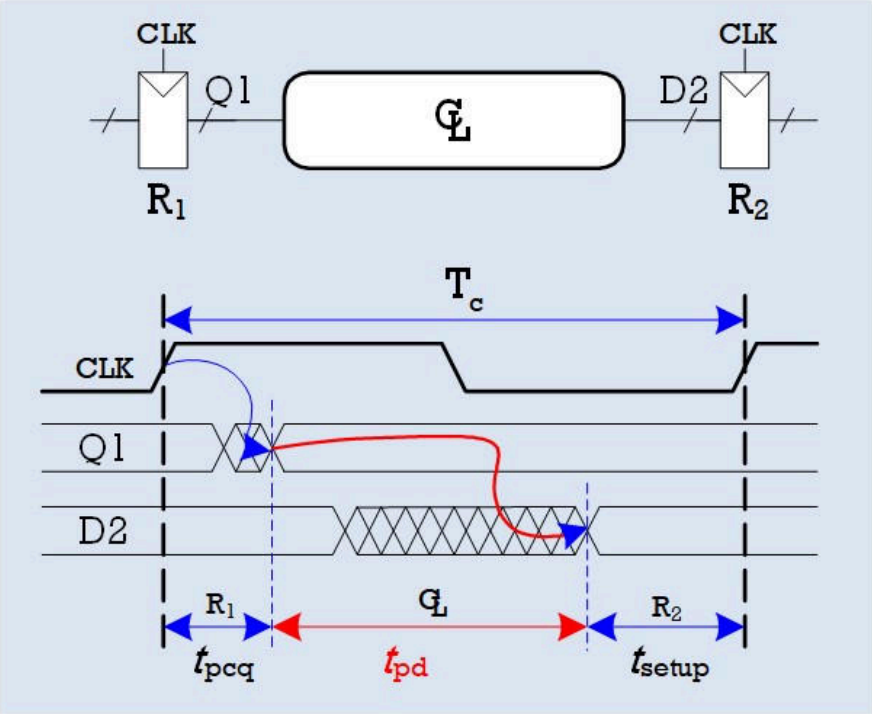


$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$
$$t_{pd} \leq$$

在商业设计中，Tc一般是由研发总监或是市场部提出的，以确保产品的竞争性；t_{pcq}和t_{setup}是由制造工艺确定的，称为时序开销；那么只有t_{pd}是设计师可以控制的。

雨课堂 Rain Classroom

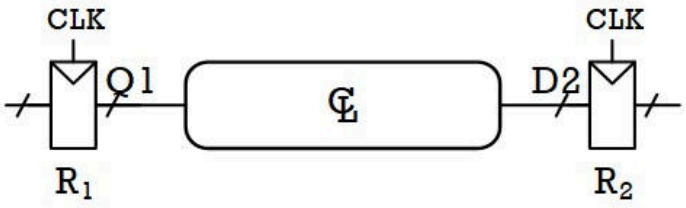
建立时间的时序约束



$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$
$$t_{pd} \leq T_c - (t_{pcq} + t_{setup})$$

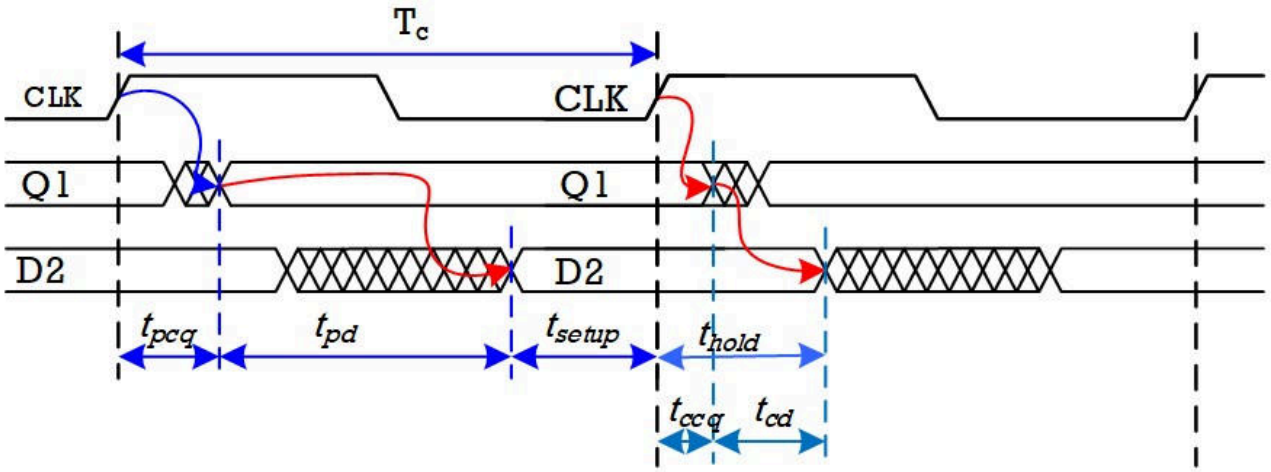
为了电路能在一个合理的时间控制下，安全稳定的运行，可以通过两个方法解决：一是增加Tc；另一个是重新设计组合逻辑部分以缩短传输延迟 t_{pd}时间。

保持时间的时序约束

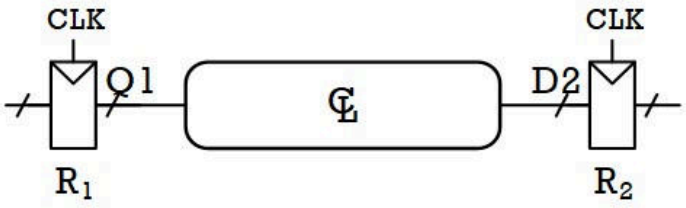


$t_{ccq} + t_{cd} > t_{hold}$

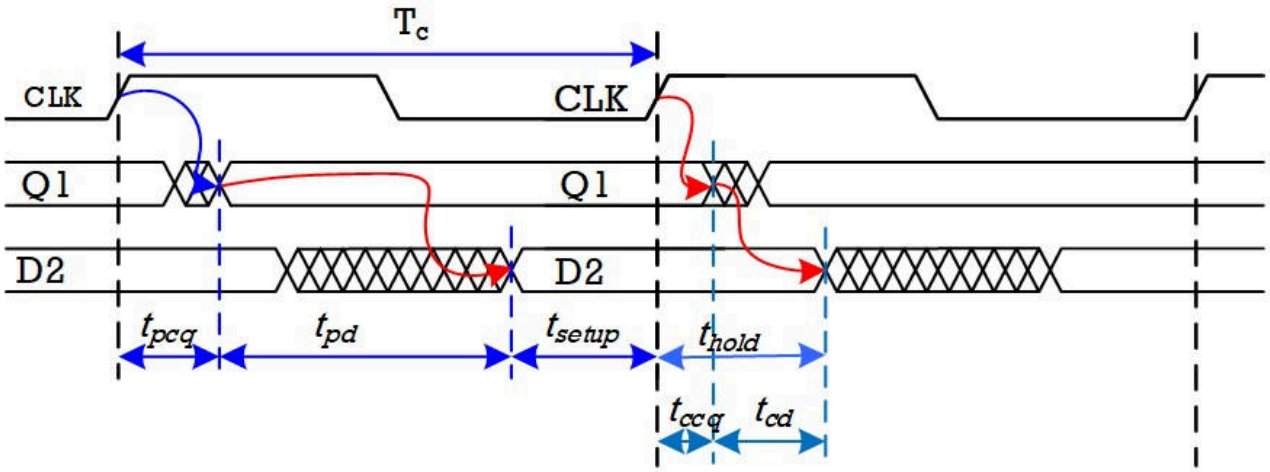
$t_{cd} >$



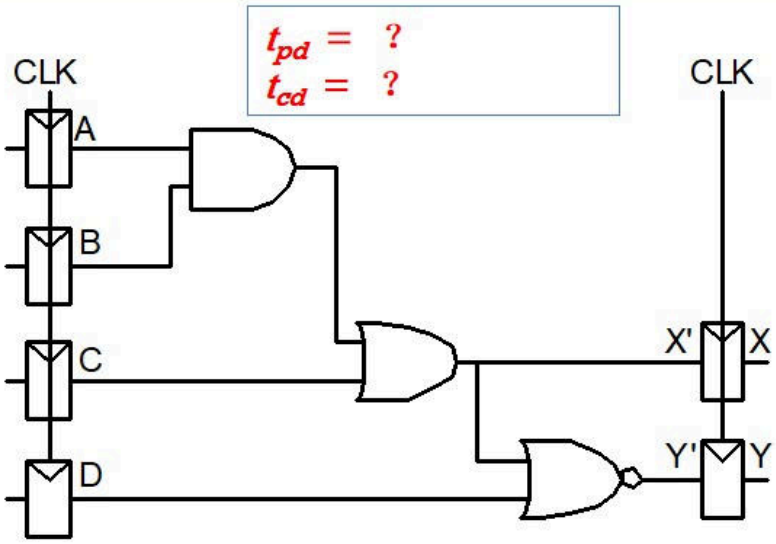
保持时间的时序约束



$$t_{ccq} + t_{cd} > t_{hold}$$
$$t_{cd} > t_{hold} - t_{ccq}$$



时序分析



时序特征

t_{ccq} = 30 ps
t_{pcq} = 50 ps
t_{setup} = 60 ps
t_{hold} = 70 ps

每个寄存器

t_{pd} = 35 ps
t_{ca} = 25 ps

每个逻辑门

建立时间的时序约束:

$T_c \geq$

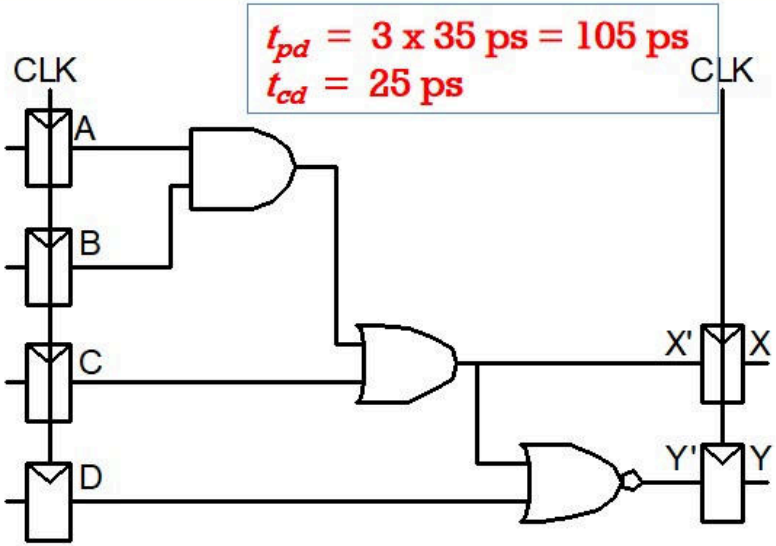
$f_c =$

保持时间的时序约束:

$t_{ccq} + t_{cd} > t_{hold}$?

雨课堂 Rain Classroom

时序分析



时序特征

t_{ccq} = 30 ps
t_{pcq} = 50 ps
t_{setup} = 60 ps
t_{hold} = 70 ps

每个寄存器

t_{pd} = 35 ps
t_{ca} = 25 ps

每个逻辑门

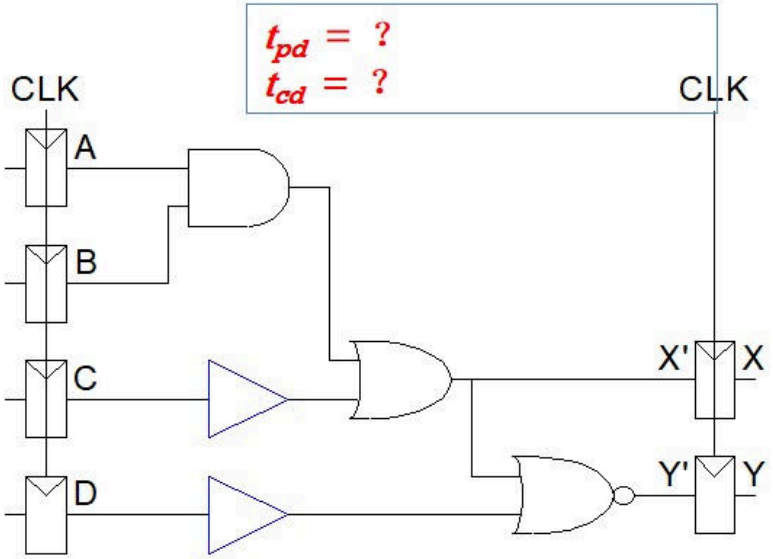
建立时间的时序约束:

$T_c \geq (50 + 105 + 60)$ ps = 215 ps
$f_c = 1/T_c = 4.65$ GHz

保持时间的时序约束:

$t_{ccq} + t_{cd} > t_{hold}$?
(30 + 25) ps > 70 ps ? No!

时序分析



时序特征

t_{ccq} = 30 ps
t_{pcq} = 50 ps
t_{setup} = 60 ps
t_{hold} = 70 ps

每个寄存器

t_{pd} = 35 ps
t_{ca} = 25 ps

每个逻辑门

建立时间的时序约束:

$T_c \geq$

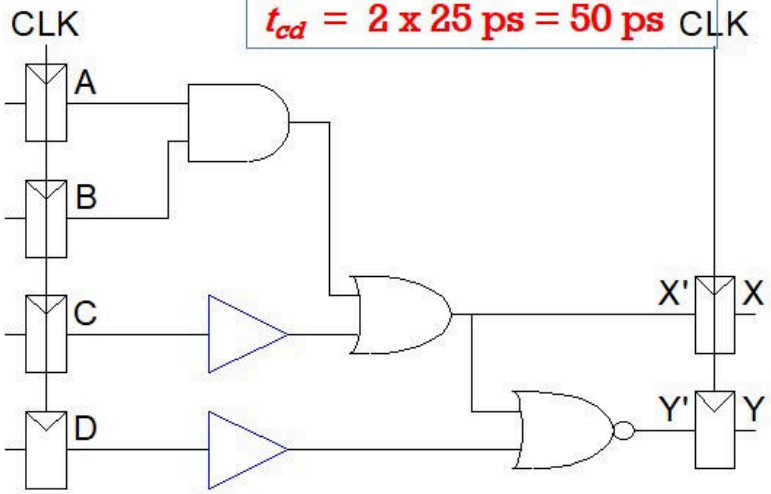
$f_c =$

保持时间的时序约束:

$t_{ccq} + t_{cd} > t_{hold}$?

时序分析

t_{pd} = 3 x 35 ps = 105 ps
t_{cd} = 2 x 25 ps = 50 ps



时序特征

t_{ccq} = 30 ps
t_{pcq} = 50 ps
t_{setup} = 60 ps
t_{hold} = 70 ps

每个寄存器

t_{pd} = 35 ps
t_{ca} = 25 ps

每个逻辑门

建立时间的时序约束:

$T_c \geq (50 + 105 + 60)$ ps = 215 ps
$f_c = 1/T_c = 4.65$ GHz

保持时间的时序约束:

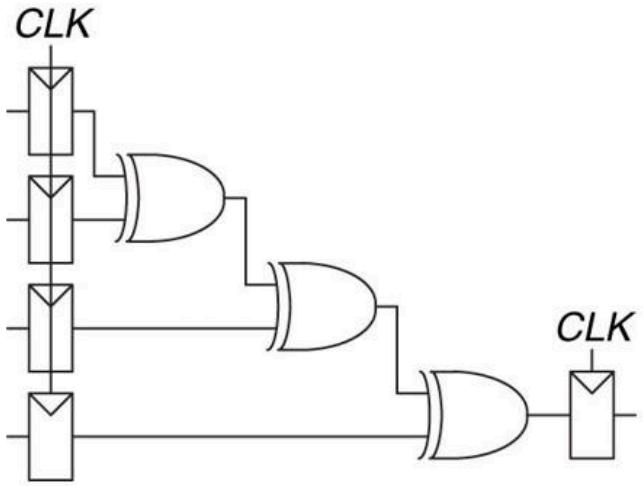
$t_{ccq} + t_{cd} > t_{hold}$?
(30 + 50) ps > 70 ps ? Yes!

慕课堂练习

- Ben设计了一个电路以实现寄存器型 4-input 异或/XOR 计算。每个2-input 异或门的传输延迟是 100 ps，最小延迟是 55 ps。每个触发器的建立时间是60 ps, 保持时间是20 ps，寄存器的传输延迟（clock-to-Q maximum delay）是 70 ps, 最小延迟（clock-to-Q minimum delay）是 50 ps。

(a) 该电路的最高工作频率（时钟频率）是多少?

(b) 怎样改进电路以提高工作频率，使之更快?



单选题 1分

关于建立时间（Setup time）的定义，以下描述正确的是?

A 指时钟边沿之后，输出Q从不稳定到稳定所需的时间

B 指时钟边沿到来之前，输入数据必须保持稳定的最短时间

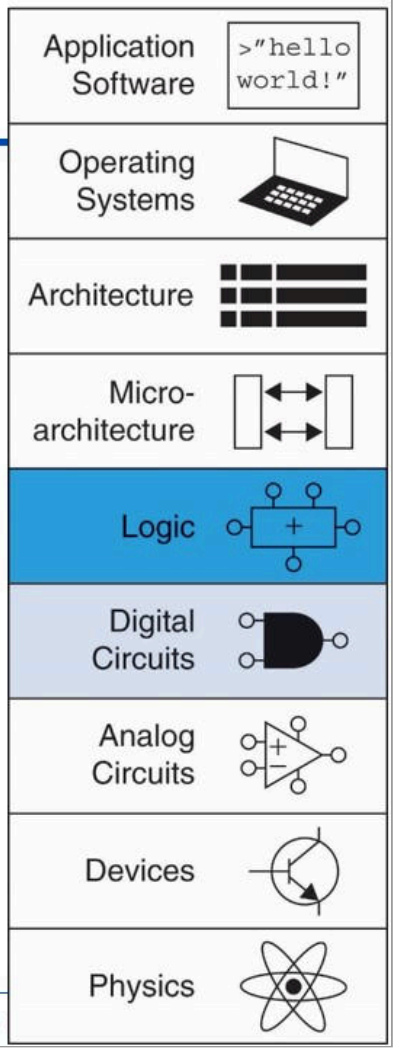
C 指输入数据在时钟沿前后必须保持稳定的总时间，即孔径时间

D 指时钟边沿之后，输入数据仍需保持稳定的最短时间

雨课堂 Rain Classroom

Chapter 3 :: Topics

- 引言
- 锁存器 (Latches) 和 触发器 (Flip-Flops)
- 寄存器 (Register) 和 寄存器文件 (Register File)
- 同步逻辑设计 (Synchronous Logic Design)
- 有限状态机 (Finite State Machines)
- 时序逻辑的时序 (Timing of Sequential Logic)
- **并行 (Parallelism)**



并行

- 系统的速度可以用**延迟**和通过系统的**任务吞吐量**来衡量。
- 前面我们讲的内容涉及到延迟等问题的讨论 。下面来讨论一下任务吞吐量：在同一时间内处理多个任务可以提高吞吐量，这就是并行。

- 并行有两种形式：**空间并行**和**时间并行**。
  - 空间并行(**Spatial parallelism**)：提供多个相同的硬件，这样多个任务就可以在同一时间一起处理。
  - 时间并行(**Temporal parallelism**)：一个任务被分成多个阶段，也称为**流水线**，就像工厂的生产流水装配线。

并行涉及的几个术语

- 任务(Token): 定义为一组输入，该组输入经过处理后能产生一组输出。
- 延迟(Latency): 是一个任务从开始到结束所需要的时间。
- 吞吐量(Throughput): 是系统单位时间内产生任务的数量

并行化可以增加吞吐量!

“并行” 示例

- Ben 决定举行一个饼干party来庆祝交通灯控制系统的成功安装，做饼干的步骤是：
  - 5 minutes 揉制饼干胚
  - 15 minutes 烘烤
- 不用任何并行处理的话，Ben烤饼干任务的延迟时间和吞吐量是多少？
  - 吞吐量以一个小时为单位计算

雨课堂 Rain Classroom

“并行” 示例

- Ben 决定举行一个饼干party来庆祝交通灯控制系统的成功安装，做饼干的步骤是：
  - 5 minutes 揉制饼干胚
  - 15 minutes 烘烤

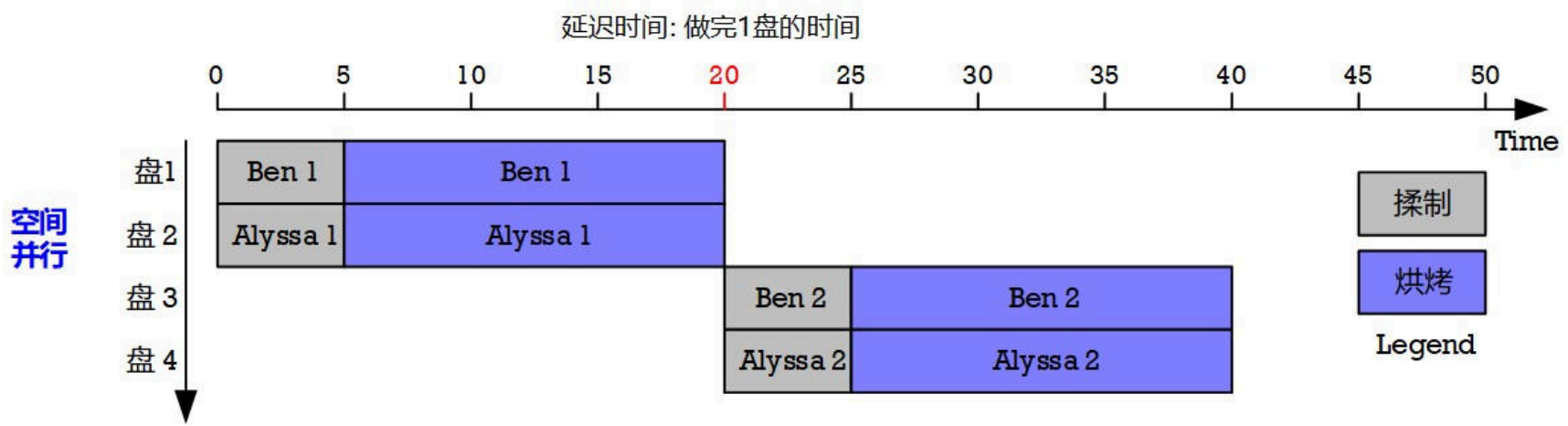
- 不用任何并行处理的话，Ben烤饼干任务的延迟时间和吞吐量是多少？
  - **延迟时间Latency** = 5 + 15 = 20 minutes = 1/3 hour
  - **吞吐量Throughput** = 1盘/ (1/3h)= 3 盘/小时

“并行” 示例

- 如果Ben 使用并行处理，那么延迟时间和吞吐量是多少?
  - **空间并行(Spatial parallelism)**:
    - Ben 求助 Allysa, 让 Allysa 带来她的烤箱
    - 增加了硬件部件（烤箱）
  - **时间并行(Temporal parallelism)**:
    - 将任务分两阶段: 制胚 和 烘烤
    - Ben使用两个烤盘(烤盘不算增加硬件部件)
    - 当前一盘在烘烤的时候，Ben就开始做下一盘的饼干胚(流水线)

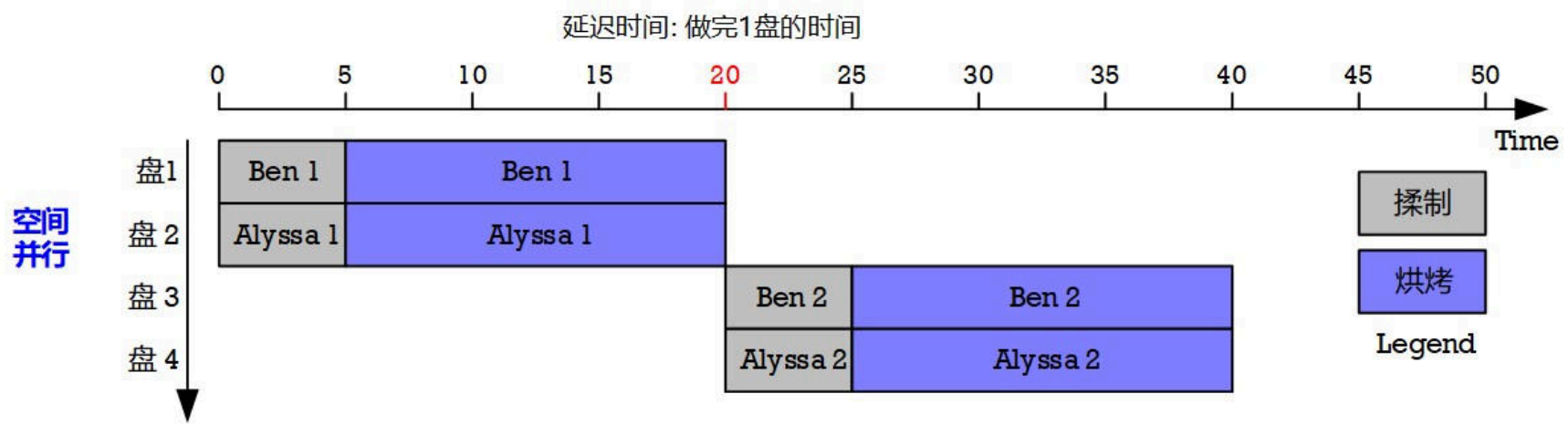
雨课堂 Rain Classroom

空间并行 Spatial Parallelism



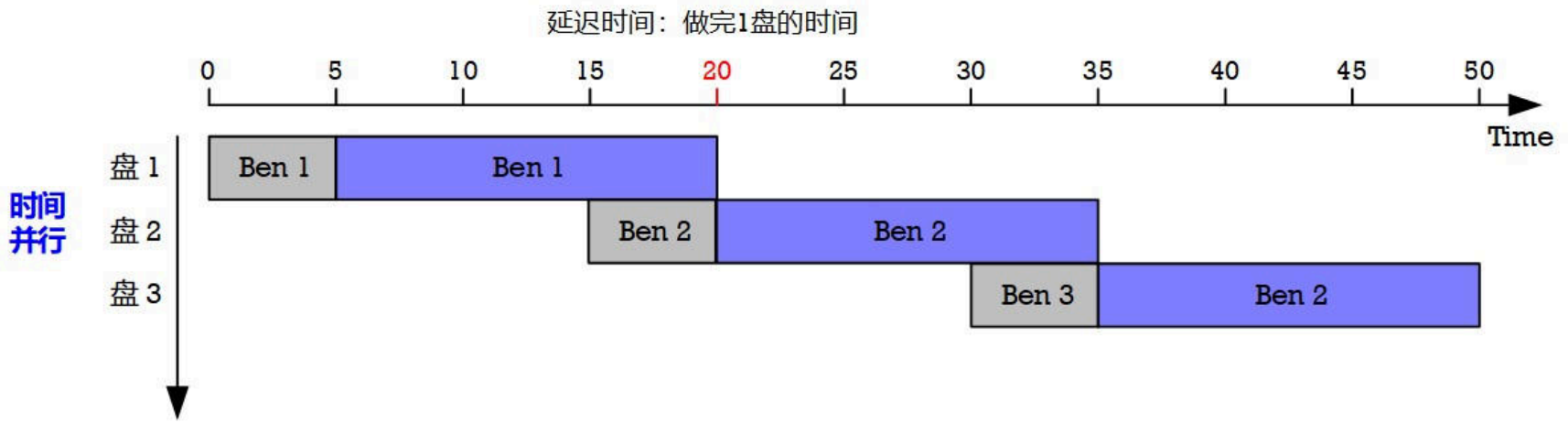
- Latency = ?
- Throughput = ?

空间并行 Spatial Parallelism



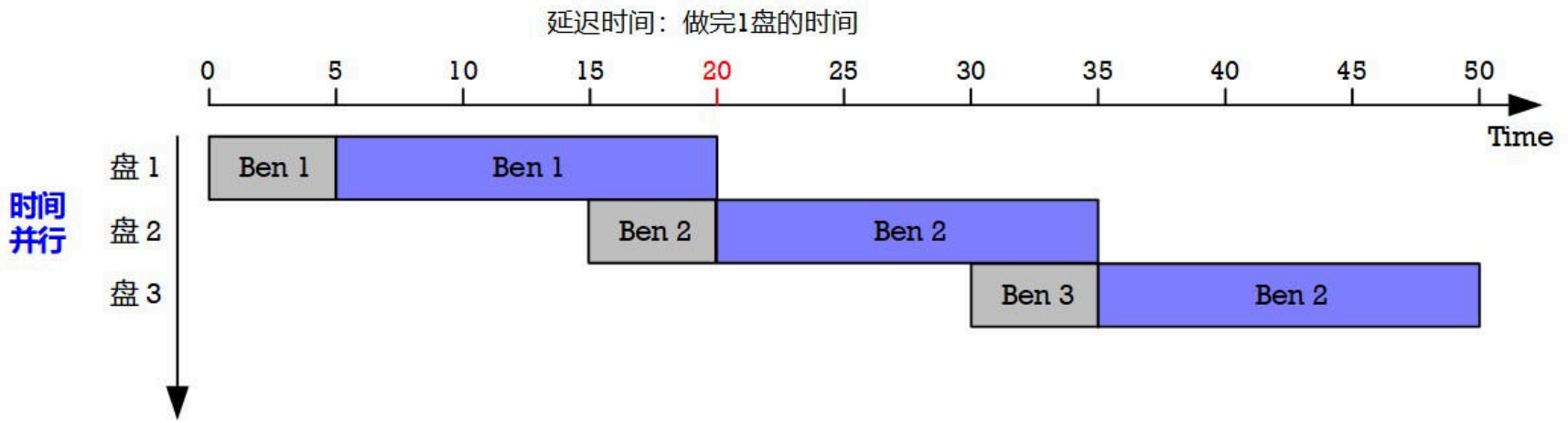
- Latency = 5 + 15 = 20 minutes = 1/3 hour
- Throughput = 2盘/ (1/3h) = 6盘/小时

时间并行Temporal Parallelism（流水线）



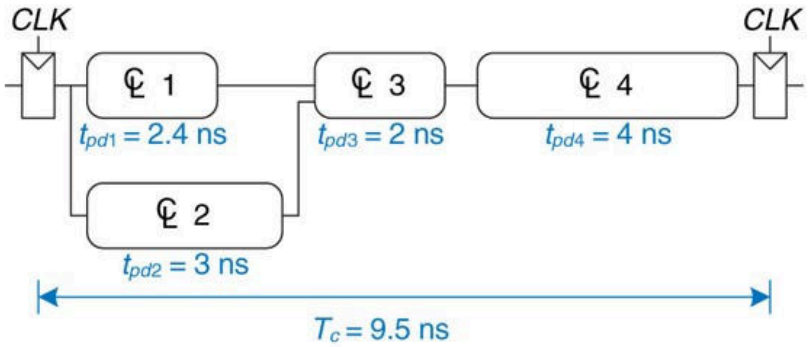
- Latency = ?
- Throughput = ?

Temporal Parallelism (流水线)



- Latency = 5 + 15 = 20 minutes = 1/3 hour
- Throughput = $^{1\text{盘}}/_{1/4\text{ h}}$ = 4 盘/小时
- 如果既使用空间并行也使用时间并行，吞吐量是 8 盘/小时

没有流水线的电路



Timing Characteristics

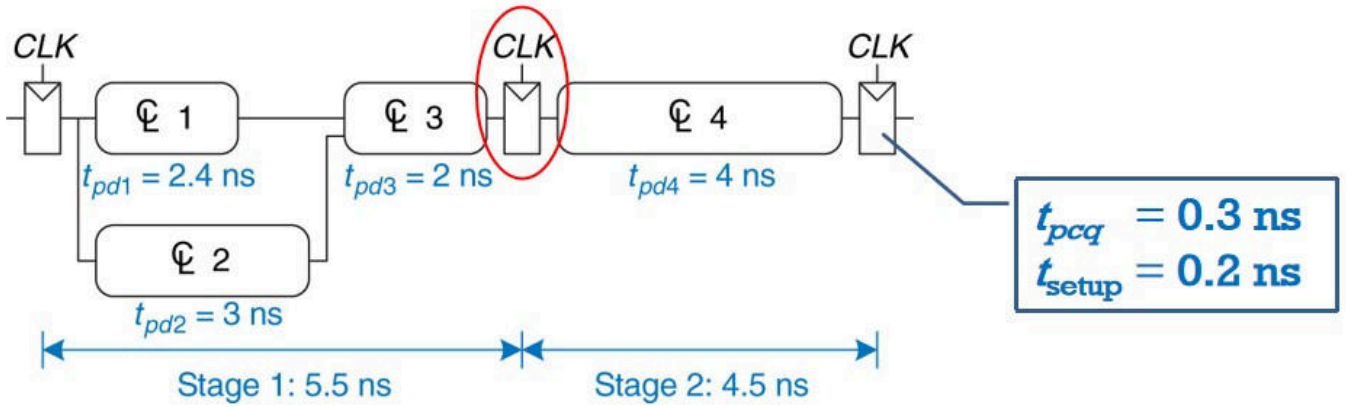
t_{pcq} = 0.3 ns

t_{setup} = 0.2 ns

- 上图给出了一个**没采用流水线**的电路示例.
  - 电路的**延迟时间**是 9.5 ns， 所以**吞吐量**是1/9.5ns = 105 MHz.

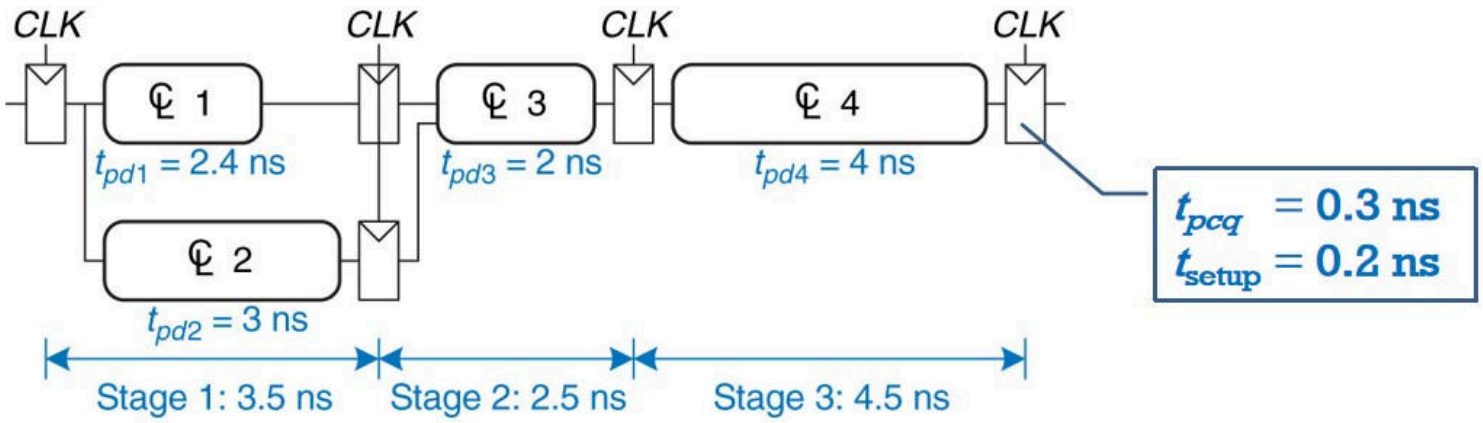
雨课堂 Rain Classroom

增加时间并行（流水线）



- Tc = 5.5 ns.
- **延迟时间**是两个时钟周期(clock cycles), 即 **11 ns**.
- **吞吐量**是1/5.5ns = **182 MHz**.

再增加时间并行（流水线）



- Tc = 4.5 ns.
- **延迟时间**是三个时钟周期(clock cycles), 即 13.5 ns.
- **吞吐量**是 1/4.5 ns = 222 MHz.

➢ 再一次增加流水级数改进了吞吐量，但是增加了延迟时间。.

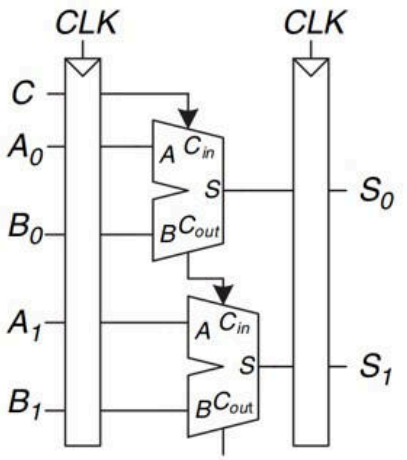
时间并行（流水线）

- 没有流水的电路.
  - Tc = 9.5 ns. 电路的延迟时间是 9.5 ns
  - 吞吐量是 1/9.5 ns = 105 MHz.
- 带有两级流水的电路.
  - Tc = 5.5 ns. 延迟时间是2个时钟周期, 即 11 ns.
  - 吞吐量是 1/5.5 ns = 182 MHz.
- 带有三级流水的电路.
  - Tc = 4.5 ns. 延迟时间是3个时钟周期, 即 13.5 ns.
  - 吞吐量是 1/4.5 ns = 222 MHz.
- 理想的流水线将使吞吐量精确地成倍增长，而不会增加任务的延迟。 但是在实际电路中并非如此。 之所以会出现差异，是因为无法将电路划分为完全相等的多个阶段，并且寄存器会带来更多的延迟开销。

雨课堂 Rain Classroom

慕课堂讨论

- 你正在为一款2 位处理器设计加法器。该加法器由两个全加器级联构成：第一个全加器的进位输出作为第二个全加器的进位输入，如右图所示。加法器带有输入寄存器与输出寄存器，运算必须在单个时钟周期内完成。
- **每个全加器的传输延迟(propagation delay)参数：**
  - 从进位输入C_{in}到进位输出C_{out}或者和S：20 ps，
  - 从输入A或B到进位输出C_{out}：25 ps，
  - 从输入A或B到和S：30 ps.
- **加法器的最小延迟(contamination delay)参数：**
  - 从进位输入C_{in}到任意输出：15 ps，从输入A或B到任意输出：22 ps，
- **每个触发器（flip-flop）参数：**
  - 建立时间 t_{setup}=30 ps，保持时间 t_{hold}=10 ps
  - 时钟到 Q 传输延迟 t_{pcq}=35 ps，时钟到 Q 最小延迟 t_{ccq}=21 ps.



电路中加法器的 t_{pd} =?

多选题 1分

下列哪些因素会导致实际流水线电路无法达到理想吞吐量倍增效果？（多选）

A 各流水级逻辑负载不均衡

B 触发器的建立时间 t_{setup} 和时钟到Q延迟 t_{pcq} 引入额外开销

C 最小延迟 t_{cd} 过大导致保持时间违例

D 流水线级数过多使单级逻辑过浅，互连延迟占比上升

雨课堂 Rain Classroom

第十一堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom